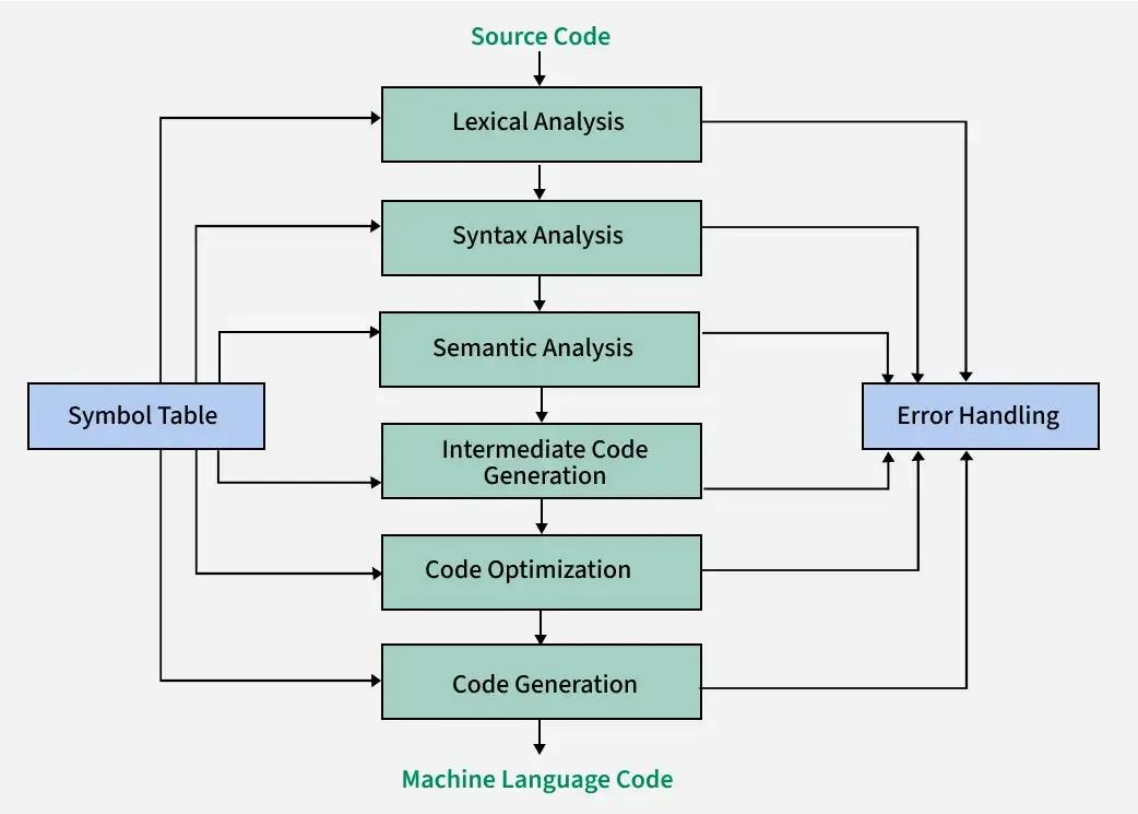
Τμήμα Μηχανικών Ηλεκτρονικών Υπολογιστών και Πληροφορικής

Μάθημα: Μεταφραστές

Αναφορά Προγραμματιστικής Εργασίας



Καραμπέρης Παναγιώτης ΑΜ:5241

Βλατάς Γεώργιος ΑΜ:5190

Περιεχόμενα

Κεφάλαιο 1ο : Εισαγωγή: 3

Κεφάλαιο 2ο: Υλοποίηση Μεταφραστή 4

2.1 Λεκτικός Αναλυτής: 4

2.2 Συντακτικός Αναλυτής: 7

2.3 Ενδιάμεσος Κώδικας: 15

2.4 Πίνακας Συμβόλων: 21

2.5 Σημασιολογική Ανάλυση: 24

2.6 Παραγωγή Τελικού Κώδικα: 25

Κεφάλαιο 3ο: Παραδείγματα ορθής λειτουργείας 28

3.1demo1 28

3.2 demo2 33

Κεφάλαιο 1ο: Εισαγωγή:

Στα πλαίσια της παρούσας εργασίας κληθήκαμε να υλοποιήσουμε έναν compiler για τη γλώσσα Greek++, μια εκπαιδευτική γλώσσα προγραμματισμού. Η εργασία καλύπτει όλα τα βασικά στάδια της μεταγλώττισης, ξεκινώντας από τη λεκτική και συντακτική ανάλυση του πηγαίου κώδικα, έως τη δημιουργία ενδιάμεσης αναπαράστασης και τη μετατροπή τους σε κώδικα χαμηλότερου επιπέδου(assembly). Η γλώσσα Greek++ υποστηρίζει βασικές προγραμματιστικές δομές όπως μεταβλητές, συναρτήσεις, διαδικασίες, δομές ελέγχου ροής.

Κεφάλαιο 2ο: Υλοποίηση Μεταφραστή

2.1 Λεκτικός Αναλυτής:

Ο λεκτικός αναλυτής (lexical analyzer) αποτελεί το πρώτο στάδιο του μεταφραστή και είναι υπεύθυνος για τη μετατροπή της ακολουθίας χαρακτήρων της greek++ σε ακολουθία λεκτικών μονάδων (tokens). Κάθε token αντιστοιχεί σε μια συντακτικά έγκυρη μονάδα, όπως αριθμοί, τελεστές, αναγνωριστικά, λέξεις-κλειδιά, κ.ά.

Ο λεκτικός αναλυτής ξεκινά τη λειτουργία του με την ανάγνωση του αρχείου πηγαίου κώδικα που του δίνεται ως είσοδος. Το αρχείο διαβάζεται χαρακτήρα προς χαρακτήρα, με αποτέλεσμα να είναι δυνατή σταδιακή κατασκευή των λεκτικών μονάδων (tokens).

Η γλώσσα greek++ υποστηρίζει τους ακόλουθους χαρακτήρες:

- Ελληνικούς χαρακτήρες (Α-Ω και α-ω)
- Λατινικούς χαρακτήρες (A-Z και a-z)
- Ακέραιους Αριθμούς
- Μαθηματκούς Τελεστές (+, -, *, /)
- Σχεσιακούς Τελεστές (=, <, >, <=, >=, <>)
- Σύμβολο Ανάθεσης (:=)
- Σύμβολα Ομαδοποίησης (‘(’, ‘)’, ‘[‘, ‘]’)
- Σύμβολα Οριοθέτησης (‘,’, ‘;’)
- Σύμβολο Αναφοράς (%)
- Σχόλια (‘{‘, ‘}’)

Επιπλέον, υπάρχουν και δεσμευμένες λεξεις κλειδιά (Keywords), οι οποίες δεν μπορούν να χρησιμοποιηθουν ως **identifiers**:

['**πρόγραμμα**', '**δήλωση**', '**εάν**', '**τότε**', '**αλλιώς**', '**εάν_τέλος**', '**επανάλαβε**', '**μέχρι**', '**όσο**', '**όσο_τέλος**', '**για**', '**έως**', '**με_βήμα**', '**για_τέλος**', '**διάβασε**', '**γράψε**', '**συνάρτηση**', '**διαδικασία**', '**διαπροσωπεία**', '**είσοδος**', '**έξοδος**', '**αρχή_συνάρτησης**', '**τέλος_συνάρτησης**', '**αρχή_διαδικασίας**', '**τέλος_διαδικασίας**', '**αρχή_προγράμματος**', '**τέλος_προγράμματος**', '**ή**', '**και**', '**εκτέλεσε**']

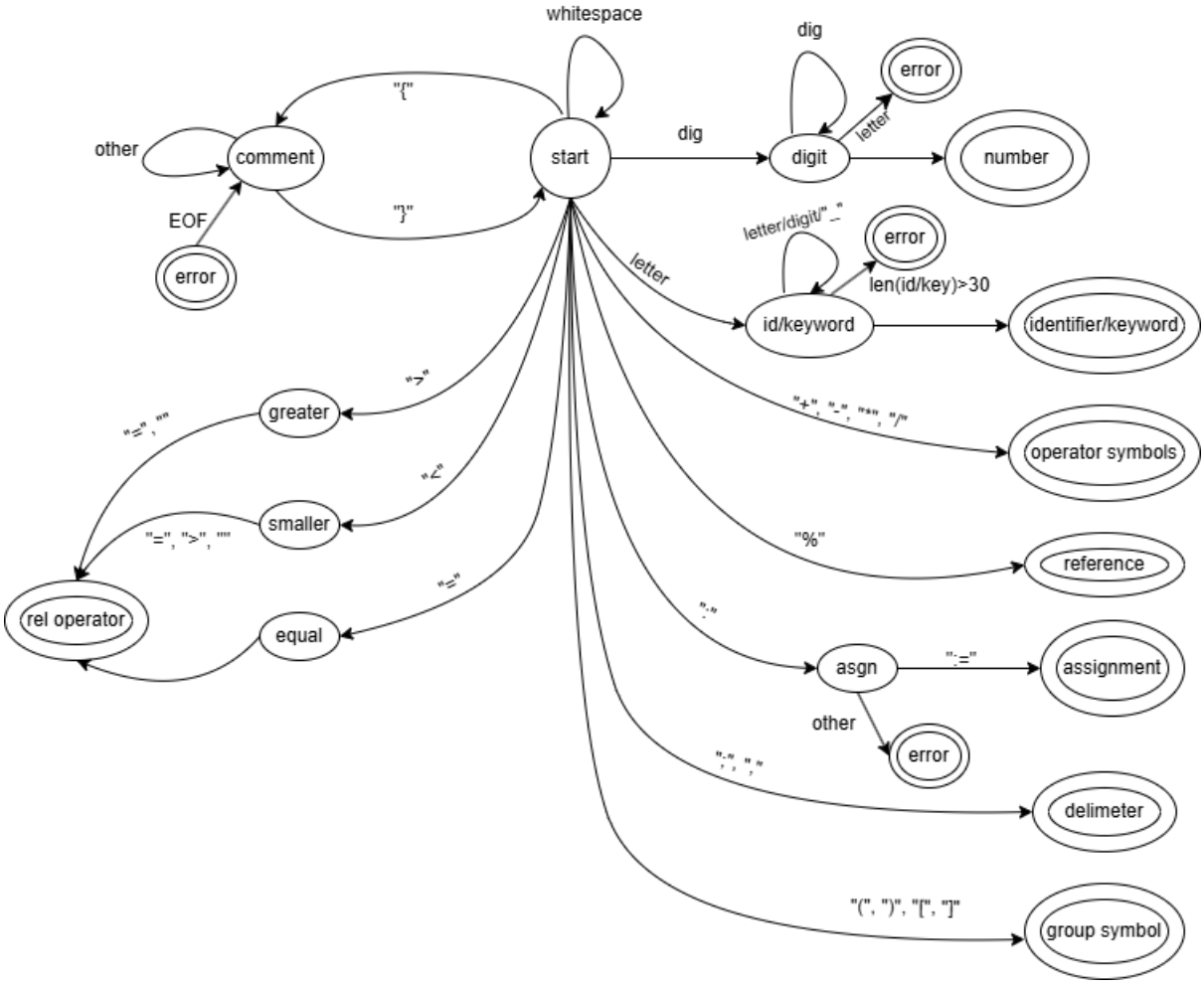
Ο λεκτικός αναλυτής ξεκινά με τον ορισμό της κλάσης Token, η οποία χρησιμοποιείται για την αναπαράσταση των λεκτικών μονάδων (tokens) που αναγνωρίζονται κατά την ανάλυση της εισόδου. Η κλάση Token διαθέτει έναν constructor (μέθοδο **__init__**) που δέχεται τρία ορίσματα:

- **tokenType**: δηλώνει τον τύπο του token, π.χ. αριθμός, αναγνωριστικό, λέξη-κλειδί, τελεστής, κ.ά.
- **token**: είναι η συμβολοσειρά που αντιστοιχεί στο ίδιο το token όπως αυτό διαβάστηκε από το αρχείο εισόδου.

- **line_counter**: δείχνει τη γραμμή του αρχείου στην οποία εντοπίστηκε το token, ώστε να είναι δυνατός ο εντοπισμός σφαλμάτων.

Η κλάση περιλαμβάνει επίσης τη μέθοδο __**str**__, η οποία βοηθάει στην εκτύπωση των αντικειμένων, εμφανίζοντας τον τύπο, την τιμή και τη γραμμή του κάθε token.

Η λειτουργία του βασίζεται στη χρήση του ακόλουθου πεπερασμένου αυτόματου , το οποίο επιτρέπει τη γραμματική αναγνώριση και τη διαχείριση των εισόδων μέσω μεταβάσεων μεταξύ καταστάσεων.



<u>Περιγραφή Αυτόματου:</u>

- Κατάσταση **start**:
  Από την κατάσταση start, το αυτόματο ελέγχει τον πρώτο χαρακτήρα και πηγαίνει στην αντίστοιχη κατάσταση ανάλογα με το είδος του, ενω άμα διαβάζει whitespace παραμένει στο start.

- Κατάσταση **digit**:
  Αν ο επόμενος χαρακτήρας είναι ψηφίο, παραμένει στο digit, ενώ αν βρεθεί γράμμα μετά από κάποιο ψηφίο τότε τυπώνουμε το αντίστοιχο error. Διαφορετικα, αν διαβάσει κενό χαρακτήρα ή newline επιστρέφει στην κατάσταση **start** και αναγνωρίζεται το token ως number.

- Κατάσταστη **id/keyword**:
  Αν ο επόμενος χαρακτήρας είναι γράμμα, ψηφίο ή '_', παραμένει στην ίδια κατάσταση. Αν ξεπεράσουμε τους 30 χαρακτήρες τότε τυπώνουμε το αντίστοιχο error. 'Οταν ολοκληρωθεί η λέξη ελέγχουμε αν είναι λέξη κλειδί ή identifier, αναγνωρίζουμε το token και επιστρέφουμε στην κατάσταση **start.**

- Καταστάσεις **Τελεστών και Συμβόλων:**
  - <u>Μαθηματικοί Τελεστές</u>: Πηγαίνουν απευθείας στην κατάσταση **operator symbols** και φτιάχνεται το token

  - <u>Σχεσιακοί Τελεστές</u>:
    Αν διαβάσει '=' μεταβαίνει στην κατάσταση **rel operator**.

    Αν διαβάσει '<' έχουμε ελέγχουμε αν ακολουθεί '=', ή '>' και μεταβαίνει στην κατάσταση **rel operator.**

    Αν διαβάσει '>' έχουμε ελέγχουμε αν ακολουθεί '=', και μεταβαίνει στην κατάσταση **rel operator.**

  - <u>Σύμβολο Ανάθεσης:</u>
    Αν διαβάσει ':' , μεταβαίνει στην κατάσταση **asgn**. Εάν μετά ακολουθεί '=' μεταβαίνει στην κατάσταση **assignment.** Εάν ωστόσο, μετα το ':' δεν ακολουθεί '=' τότε τυπώνουμε το αντίστοιχο error.

  - <u>Σύμβολο Αναφοράς:</u>
    Αν διαβάσει '%' μετβαίνει στην κατάσταση **reference**.

  - <u>Σύμβολα Οριοθέτησης:</u>
    Πηγαίνουν απευθείας στην κατάσταση **delimiter**.

  - <u>Σύμβολα Ομαδοποίησης:</u>
    Πηγαίνουν απευθείας στην κατάσταση **group symbol**.

- Κατάσταστη **comment**:

  Ξεκινάει με το που διάβασει '{' και παραμένει στη κατάσταση **comment** μέχρι να βρεθεί κλείσιμο σχολίων '}'. Εάν δεν κλείσει ποτέ τυπώνουμε το αντίστοιχο error.

Συνολική Λειτουργία:

Το αυτόματο ξεκινά από μία αρχική κατάσταση και μεταβαίνει σε διάφορες ενδιάμεσες και τελικές καταστάσεις, ανάλογα με τους χαρακτήρες που διαβάζει. Κάθε τελική κατάσταση αντιστοιχεί σε έναν τύπο λεκτικής μονάδας (token), όπως αριθμός ή λέξη κλειδί. Όταν το αυτόματο φτάσει σε μία τέτοια τελική κατάσταση, αναγνωρίζει το αντίστοιχο token και το προσθέτει στη λίστα **tokens**. Η λίστα αυτή είναι πολύ σημαντική, καθώς θα χρησιμοποιηθεί αργότερα από τον Συντακτικό Αναλυτή για να κατανοήσει τη γραμματική του προγράμματος.

2.2 Συντακτικός Αναλυτής:

Ο συντακτικός αναλυτής (Parser) είναι το δεύτερο στάδιο που κληθήκαμε να υλοποιήσουμε και έχει ως στόχο να επαληθεύσει τη συντακτική ορθότητα της εισόδου με βάση τη γραμματική της γλώσσας. Αξιοποιεί την λίστα με τα tokens που έχει παραχθεί από τον λεκτικό αναλυτή και ελέγχει αν η σειρά και ο τύπος τους συνάδουν με τους κανόνες γραμματικής που ορίζει η γλώσσα. Αν συναντηθεί κάποιο συντακτικό σφάλμα, ο parser το εντοπίζει και το αναφέρει με κατάλληλο μήνυμα. Ο συντακτικός αναλυτής που φτιάξαμε ακολουθεί μια προσέγγιση καθοδικής αναδρομής, όπου για κάθε συντακτικό κανόνα υλοποιήσαμε μια ξεχωριστή μέθοδο.

Η υλοποίησή μας ξεκινά με τον ορισμό της κλάσης Parser, η οποία είναι υπεύθυνη για τη διαχείριση της συντακτικής ανάλυσης. Ο constructor (μέθοδος __init__) δέχεται ως είσοδο την λίστα **token** που δημιουργήθηκε από τον λεκτικό αναλυτή και αρχικοποιεί:

- Τον δείκτη **current_token_index** στην τιμή 0, ο οποίος δείχνει στο πρώτο token της λίστας.
- Το πεδίο current_token, στο οποίο κρατάμε το τρέχον token που εξετάζουμε.

Έπειτα, φτιάχνουμε δυο βοηθητικές μεθόδους:

- **match_token:** Η μέθοδος match_token ελέγχει αν το τρέχον token έχει τον αναμενόμενο τύπο (expected_type) και, προαιρετικά, την αναμενόμενη τιμή (expected_value). Πριν τον έλεγχο, παραλείπει τυχόν σχόλια από την είσοδο. Αν το token δεν είναι αυτό που αναμενόταν, τυπώνουμε SyntaxError με μήνυμα που περιγράφει το σφάλμα και τη γραμμή στην οποία εντοπίστηκε.
- **next_token:** Η next_token προχωρά στο επόμενο token στη λίστα. Αν δεν υπάρχουν άλλα tokens, το current_token γίνεται None, σηματοδοτώντας το τέλος της εισόδου.

Παρκάτω παραθέτουμε την γραμματική της greek++, με βάση της οποίας υλοποιήσαμε κάθε μέθοδο που περιγράφει και έναν συντακτικό κανόνα.

Γραμματική της greek++

```
program            :  'πρόγραμμα' ID programblock
                   ;

programblock       :  declarations subprograms
                        'αρχή_προγράμματος' sequence 'τέλος_προγράμματος'
                   ;

declarations       :  ( 'δήλωση' varlist )*
                   |
                   ;

varlist            :  ID ( ',' ID )*
                   ;

subprograms        :  ( func | proc )*
                   ;

func               :  'συνάρτηση' ID '(' formalparlist  ')' funcblock
                   ;

proc               :  'διαδικασία' ID '(' formalparlist  ')' procblock
                   ;

formalparlist      :  varlist
                   |
                   ;

funcblock          :  'διαπροσωπεία' funcinput funcoutput declarations
                        subprograms
                        'αρχή_συνάρτησης' sequence 'τέλος_συνάρτησης'
                   ;

procblock          :  'διαπροσωπεία' funcinput funcoutput declarations
                        subprograms
                        'αρχή_διαδικασίας' sequence 'τέλος_διαδικασίας'
                   ;

funcinput          :  'είσοδος' varlist
                   |
                   ;

funcoutput         :  'έξοδος' varlist
                   |
                   ;
```

```
sequence            :   statement ( ';' statement )*
                    ;

statement           :   assignment_stat
                        |   if_stat
                        |   while_stat
                        |   do_stat
                        |   for_stat
                        |   input_stat
                        |   print_stat
                        |   call_stat
                        ;

assignment_stat     :   ID ':=' expression
                    ;

if_stat             :   'εάν' condition 'τότε' sequence elsepart 'εάν_τέλος'
                    ;

elsepart            :   'αλλιώς' sequence
                    |
                    ;

while_stat          :   'όσο' condition 'επανάλαβε' sequence 'όσο_τέλος'
                    ;

do_stat             :   'επανάλαβε' sequence 'μέχρι' condition
                    ;

for_stat            :   'για' ID ':=' expression 'έως' expression  step
                          'επανάλαβε' sequence 'για_τέλος'
                    ;

step                :   'με_βήμα' expression
                    |
                    ;

print_stat          :   'γράψε' expression
                    ;

input_stat          :   'διάβασε' ID
                    ;

call_stat           :   'εκτέλεσε' ID idtail
                    ;
```

```
idtail              :   actualpars
                    |
                    ;

actualpars          :   '(' actualparlist ')'
                    ;

actualparlist       :   actualparitem ( ',' actualparitem )*
                    |
                    ;

actualparitem       :   expression |  '%'ID
                    ;

condition           :   boolterm ( 'ή' boolterm )*
                    ;

boolterm            :   boolfactor ( 'και' boolfactor )*
                    ;

boolfactor          :   'όχι' '[' condition ']'
                    |   '[' condition ']'
                    |   expression relational_oper expression
                    ;

expression          :   optional_sign term ( add_oper term )*
                    ;

term                :   factor  ( mul_oper factor )*
                    ;

factor              :   INTEGER
                    |   '(' expression ')'
                    |   ID idtail
                    ;

relational_oper     :   '=' | '<=' | '>=' | '<>' | '<'  | '>'
                    ;

add_oper            :   '+' | '-'
                    ;

mul_oper            :   '*' | '/'
                    ;

optional_sign       :   add_oper
                    |
                    ;
```

- **program()**: Η συνάρτηση program υλοποιεί τον συντακτικό κανόνα για την αρχή ενός προγράμματος. Ελέγχει ότι η είσοδος ξεκινά με τη λέξη-κλειδί πρόγραμμα, ακολουθούμενη από ένα έγκυρο αναγνωριστικό (το όνομα του προγράμματος). Στη συνέχεια καλεί τη **program_block**, η οποία περιέχει το κύριο σώμα του προγράμματος.

- **identifier()**: Η identifier είναι μια βοηθητική συνάρτηση που χρησιμοποιείται σε πολλά σημεία του parser για να διασφαλίσει ότι το επόμενο token είναι τύπου. Αν το token είναι όντως αναγνωριστικό, επιστρέφεται το ονομά του, εάν δεν είναι τότε εμφανίζεται το αντίστοιχο συντακτικό σφάλμα. Η συνάρτηση αυτή εξασφαλίζει την ορθότητα της σύνταξης σε όλα τα σημεία όπου απαιτείται ονομασία, όπως σε προγράμματα, μεταβλητές και συναρτήσεις.

- **program_block()**: Η program_block αντιστοιχεί στον συντακτικό κανόνα που ορίζει το κυρίως σώμα ενός προγράμματος. Αρχικά καλεί την **declarations** και την **subprograms** για να αναλύσει τις δηλώσεις μεταβλητών και τις υπορουτίνες. Έπειτα ελέγχει ότι υπάρχει η λέξη κλειδί αρχή_προγράμματος, πριν συνεχίσει με την επεξεργασία της βασικής ακολουθίας εντολών μέσω της **sequence**. Τέλος, αναμένει την λέξη κλειδί τέλος_προγράμματος ώστε να κλείσει το μπλοκ.

- **declarations()**: Η συνάρτηση declarations είναι υπεύθυνη για την αναγνώριση δηλώσεων μεταβλητών μέσα στο πρόγραμμα. Ελέγχει εάν το επόμενο token είναι η λέξη-κλειδί δήλωση και, εφόσον αυτό ισχύει, απαιτεί να ακολουθεί μία λίστα αναγνωριστικών μέσω της **var_list**. Η συγκεκριμένη διαδικασία επανλαμβάνεται όσο υπάρχουν συνεχόμενες δηλώσεις. Σε περίπτωση που λείπει αναγνωριστικό μετά από δήλωση, προκαλείται συντακτικό σφάλμα με αντίστοιχο μήνυμα.

- **var_list()**: Η συνάρτηση var_list υλοποιεί τον συντακτικό κανόνα για τη λίστα μεταβλητών, δηλαδή για μια ακολουθία αναγνωριστικών που δηλώνονται μαζί, διαχωρισμένα με κόμμα. Αρχικά, αναμένει τουλάχιστον ένα αναγνωριστικό, εάν όμως αυτό δεν υπάρχει, τότε προκαλείται συντακτικό σφάλμα. Η διαδικασία αυτή επαναλαμβάνεται για κάθε επιπλέον αναγνωριστικό που ακολουθεί, εφόσον προηγείται κόμμα. Με αυτόν τον τρόπο, διασφαλίζεται η σωστή ανάλυση των δηλώσεων πολλαπλών μεταβλητών.

- **subprograms()**: Η συνάρτηση subprograms χειρίζεται την ύπαρξη υποπρογραμμάτων μέσα στο πρόγραμμα, δηλαδή συναρτήσεων και διαδικασιών. Όσο εντοπίζονται tokens που αντιστοιχούν στις λέξεις κλειδιά συνάρτηση ή διαδικασία, καλούνται αντίστοιχα οι συναρτήσεις **func** και **proc** για να αναλύσουν κάθε μία από αυτές τις οντότητες.

- **func()**: Η συνάρτηση func αναλαμβάνει την ανάλυση της σύνταξης για τη δήλωση μιας συνάρτησης. Ξεκινά με τον έλεγχο της λέξης κλειδιού συνάρτηση και ακολουθεί η ανάγνωση του ονόματος της συνάρτησης μέσω της **identifier.** Έπειτα ελέγχεται αν ακολουθεί σωστά ανοιχτή και κλειστή παρένθεση, όπου μέσα βρίσκονται οι παράμετροι και καλούμε την **formal_par_list** για την ανάλυσή τους. Τέλος, καλείται η func_block που χειρίζεται το σώμα της συνάρτησης.

- **proc()**: Αντίστοιχα, η συνάρτηση proc εκτελεί την ίδια συντακτική ανάλυση με την func, αλλά για τη δήλωση μιας διαδικασίας. Ξεκινά με την αναγνώριση της λέξης κλειδιού διαδικασία και συνεχίζει με την ανάγνωση του αναγνωριστικού ονόματος της διαδικασίας μέσω της **identifier**. Έπειτα αναμένουμε τις παραμέτρους οι οποίες περικλείονται από παρενθέσεις. Στη συνέχεια καλούμε την **formal_par_list** για την ανάλυση των παραμέτρων και τελικά καλούμε την **proc_block** για την επεξεργασία του σώματος της διαδικασίας.

- **func_block()**: Η συνάρτηση func_block υλοποιεί τον συντακτικό κανόνα που αφορά το σώμα μιας συνάρτησης. Αρχικά ελέγχει για την λέξη κλειδί διαπροσωπεία και έπειτα καλούνται οι **func_input** και **func_output**, οι οποίες ελέγχουν αντίστοιχα τις παραμέτρους εισόδου και εξόδου. Ακολουθεί η ανάλυση των δηλώσεων μεταβλητών μέσω της **declarations**, ενώ στη συνέχεια εξετάζεται αν υπάρχουν εσωτερικά υποπρογράμματα, όπου και καλείται η **subprograms**. Στη συνέχεια, γίνεται ο έλεγχος για την έναρξη του κύριου σώματος της συνάρτησης με τη λέξη κλειδί αρχή_συνάρτησης. Το κύριο σώμα αναλύεται με τη **sequence**, και τέλος, αναμένεται η λέξη τέλος_συνάρτησης.

- **proc_block()**: Η συνάρτηση proc_block ακολουθεί την ίδια λογική με την func_block. Ανμένουμε, αρχικά την λέξη κλειδί διαπροσωπεία και μετά καλούμε τις συναρτήσεις **func_input**, **func_output**, **declarations** και **subprograms** με τον ίδιο τρόπο όπως στην fuc_block. Εφόσον γίνουν όλα αυτά, αναμένουμε την λέξη κλειδί αρχή_διαδικασίας και καλούμε την sequence. Τέλος ολοκληρώνεται η διαδικασία με την αναγνώριση του τέλος_διαδικασίας.

- **func_input()**: Η συνάρτηση func_input υλοποιεί τον συντακτικό κανόνα που αφορά την εισαγωγή παραμέτρων σε μια συνάρτηση ή διαδικασία. Αρχικά ελέγχουμε για την λέξη κλειδί είσοδος και έπειτα καλούμε την **var_list** για την ανάγνωση της λίστας μεταβλητών εισόδου.

- **func_output()**: Η συνάρτηση func_output λειτουργεί με αντίστοιχο τρόπο για την έξοδο από μια συνάρτηση ή διαδικασία. Αρχικά ελέγχουμε για την λέξη κλειδί έξοδος και έπειτα καλούμε την **var_list** για την ανάγνωση της λίστας μεταβλητών εξόδου.

- **sequence()**: Η συνάρτηση sequence αναλύει μια ακολουθία εντολών, δηλαδή ένα σύνολο από επιμέρους δηλώσεις που διαχωρίζονται με ελληνικό ερωτηματικό ';'. Αρχικά, παραλείπονται τυχόν σχόλια και αμέσως μετά καλείται η **statement** για την ανάλυση της πρώτης εντολής. Έπειτα, όσο υπάρχει ερωτηματικό, επαναλαμβάνεται η διαδικασία.

- **statement()**: Η συνάρτηση statement αποτελεί τον κεντρικό μηχανισμό ανάλυσης εντολών και αντιστοιχεί στον συντακτικό κανόνα για μία γενική εντολή. Ελέγχει τον τύπο και την τιμή του τρέχοντος token και, ανάλογα με το περιεχόμενό του, κατευθύνει την εκτέλεση στη σωστή επιμέρους συνάρτηση. Ανάλογα με την λέξη κλειδί καλείται μία απο τις παρακάτω συναρτήσεις:

  - **assignment_stat()**: Η συνάρτηση assignment_stat διαχειρίζεται εντολές ανάθεσης τιμής σε μεταβλητή. Αρχικά αναγνωρίζει το όνομα της μεταβλητής και ακολουθεί η αναγνώριση του τελεστή ανάθεσης ":=" και μετά καλούμε την **expression**.

  - **if_stat()**: Η if_stat υλοποιεί τη δομή ελέγχου εάν–τότε–αλλιώς. Αρχικά, ελέγχει ότι υπάρχει η λέξη-κλειδί εάν και καλούμε την **condition**. Στη συνέχεια απαιτείται η λέξη κλειδί τότε και ακολουθεί η κλήση της **sequence**. Αν υπάρχει το αλλιώς, καλείται η **else_stat** για να αναλυθεί και ο δεύτερος κλάδος. Η εντολή ολοκληρώνεται με τη λέξη κλειδί εάν_τέλος.

  - **else_stat()**: Η else_stat αναλύει το τμήμα του εναλλακτικού κλάδου μιας συνθήκης. Ξεκινά με τη λέξη κλειδί αλλιώς και ακολουθεί η κλήση της **sequence**.

  - **while_stat()**: Η while_stat αναλύει την δομή όσο–επανάλαβε. Αρχικά αναμένει την λέξη κλειδί **όσο** και αμέσως καλεί την **condition.** Έπειτα αναξητέι την λέξη επανάλαβε και ακολουθεί μία μια ακολουθία εντολών μέσω της **sequence**. Η εντολή κλείνει με τη λέξη-κλειδί όσο_τέλος.

- **do_stat()**: Η do_stat αναλύει τη δομή "επανάλαβε–μέχρι". Ξεκινά με τη λέξη κλειδί επανάλαβε και ακολουθεί η κλήση της **sequence**. Στη συνέχεια, αναζητεί την λέξη μέχρι και καλεί την **condition**.

- **for_stat()**: Η for_stat αναλύει τη δομή επανάληψης "για–έως–επανάλαβε". Ξεκινά με τη λέξη-κλειδί για και ακολουθεί ένα αναγνωριστικό στο οποίο ανατίθεται η αρχική τιμή της επανάληψης. Στη συνέχεια, μέσω της λέξης κλειδί έως, δηλώνεται η τελική τιμή του μετρητή. Προαιρετικά, μπορεί να καθοριστεί και ένα βήμα επανάληψης με τη λέξη κλειδί με_βήμα. Μετά τη λέξη-κλειδί επανάλαβε, ακολουθεί το σώμα της επανάληψης με την κλήση της **sequence**. Η εντολή ολοκληρώνεται με τη λέξη-κλειδί "για_τέλος".

- **print_stat()**: Η print_stat υλοποιεί τη σύνταξη της εντολής εξόδου γράψε. Αναμένει τη λέξη-κλειδί γράψε και έπειτα καλεί την **sequence.**

- **input_stat()**: Η input_stat αντιστοιχεί στη σύνταξη της εντολής εισόδου διάβασε. Ξεκινά με τη λέξη κλειδί διάβασε και αναμένει ένα αναγνωριστικό, στο οποίο θα αποθηκευτεί η τιμή που εισάγεται μέσω της **identifier**.

- **call_stat()**: Η call_stat αναλύει την εντολή κλήσης υποπρογράμματος μέσω της λέξης κλειδί εκτέλεσε. Ακολουθεί ένα αναγνωριστικό που δηλώνει τη συνάρτηση ή διαδικασία που καλείται. Εφόσον ακολουθεί παρενθετική λίστα, γίνεται ανάλυση των πραγματικών παραμέτρων μέσω της **actualpars**.

- **step()**: Η step αντιπροσωπεύει τον συντακτικό κανόνα για το “με βήμα” μέσα σε μια εντολή "για". Περιμένει την αντίστοιχη λέξη κλειδί με_βημα και στη συνέχεια καλούμε μια βοηθητικη συναρτηση που υλοποιήσαμε την **step_expression.**

- **step_expression()**: Η step_expression υλοποιεί την ανάλυση της έκφρασης βήματος μέσα σε εντολή επανάληψης με_βήμα και την φτιάξαμε για να μας διευκολύνει στην υλοποίηση της **for_stat** και συγκεκριμένα όταν έχουμε αρνητικό βήμα. Ελέγχει προαιρετικό πρόσημο στην αρχή, ακολουθούμενο από όρους με τελεστές πρόσθεσης ή αφαίρεσης. Επιστρέφει το αποτέλεσμα της έκφρασης, εφαρμόζοντας αρνητικό πρόσημο αν αυτό έχει δηλωθεί αρχικά.

- **optional_sign()**: Η optional_sign εξετάζει αν υπάρχει προαιρετικό πρόσημο (+ ή -) στην αρχή μιας αριθμητικής έκφρασης. Αν εντοπιστεί, καλεί την add_oper για να το καταχωρίσει και επιστρέφει το πρόσημο. Αν δεν υπάρχει πρόσημο, επιστρέφει None.

- **call_func()**: Η call_func χρησιμοποιείται για την κλήση συνάρτησης. Περιλαμβάνει την αναγνώριση ενός αναγνωριστικού που δηλώνει τη συνάρτηση, και εφόσον ακολουθεί παρένθεση, γίνεται επεξεργασία παραμέτρων μέσω της **actualpars**.

- **actualpars()**: Η actualpars αντιπροσωπεύει τη σύνταξη της λίστας πραγματικών παραμέτρων σε κλήση υποπρογράμματος. Ελέγχει ότι ξεκινά με αριστερή παρένθεση, αναλύει τα στοιχεία της λίστας μέσω της **actualparlist** και ολοκληρώνει με δεξιά παρένθεση.

- **actualparlist()**: Η actualparlist χειρίζεται μία λίστα παραμέτρων. Ξεκινά με ένα στοιχείο παραμέτρου και επεκτείνεται όσο υπάρχουν επιπλέον στοιχεία χωρισμένα με κόμμα, καλώντας τη συνάρτηση **actualparitem**.

- **actualparitem()**: Η actualparitem αναλύει ένα επιμέρους στοιχείο από τη λίστα παραμέτρων. Εάν πρόκειται για αναφορά μεταβλητής, αναμένεται το σύμβολο "%", ενώ σε διαφορετική περίπτωση αναλύεται ως κανονική παράμετρος μέσω της **expression**.

- **condition()**: Η condition υλοποιεί τον συντακτικό κανόνα για λογικές συνθήκες που περιλαμβάνουν τον τελεστή “ή”. Αρχικά καλεί την **boolterm** και έπειτα επαναλαμβάνει την ανάλυση για κάθε επόμενο όρο που συνδέεται με "ή", μέχρι να εξαντληθούν.

- **boolterm()**: Η boolterm υλοποιεί τον συντακτικό κανόνα για λογικούς όρους συνδεδεμένους με τον τελεστή "και" ;οπως και η condition.

- **boolfactor()**: Η boolfactor διαχειρίζεται τους βασικούς λογικούς παράγοντες μιας συνθήκης. Αυτοί μπορεί να είναι είτε απλές συγκρίσεις μεταξύ εκφράσεων είτε σύνθετες λογικές εκφράσεις σε αγκύλες, με ή χωρίς το λογικό “όχι”.

- **expression()**: Αναλύει αριθμητικές εκφράσεις που περιλαμβάνουν όρους και τελεστές πρόσθεσης/αφαίρεσης (+, -),

- **term()**: Επεξεργάζεται γινόμενα και πηλίκα μεταξύ αριθμητικών εκφράσεων. Ξεκινά με μί έκφραση και εφαρμόζει επαναληπτικά τελεστές * ή / με επόμενες εκφράσεις.

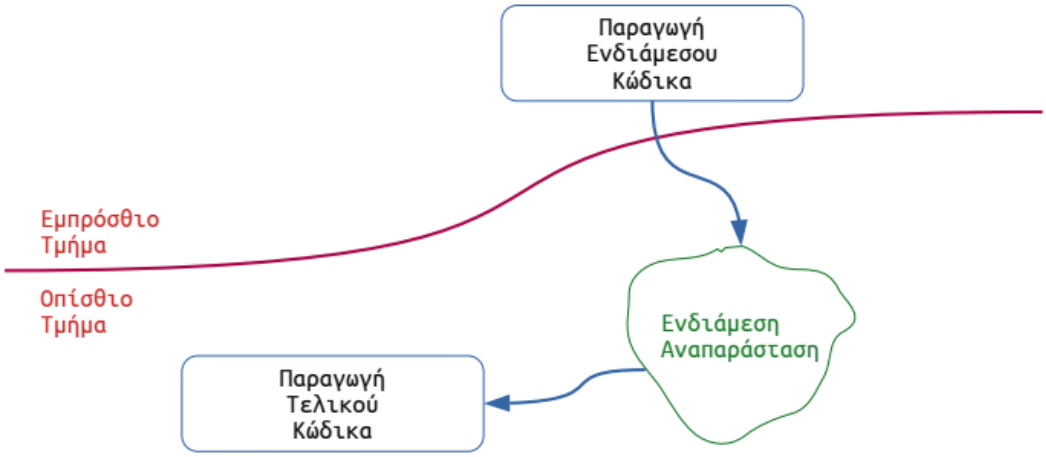
- **factor()**: Αναγνωρίζει βασικούς αριθμητικούς παράγοντες όπως αριθμούς, αναγνωριστικά, ή εκφράσεις εντός παρενθέσεων.

- **relational_oper()**: Ελέγχει και επιστρέφει έγκυρους συγκριτικούς τελεστές (=, <=, >=, <>, <, >), τυπώνοντας συντακτικό σφάλμα αν δεν βρεθεί κάποιος.

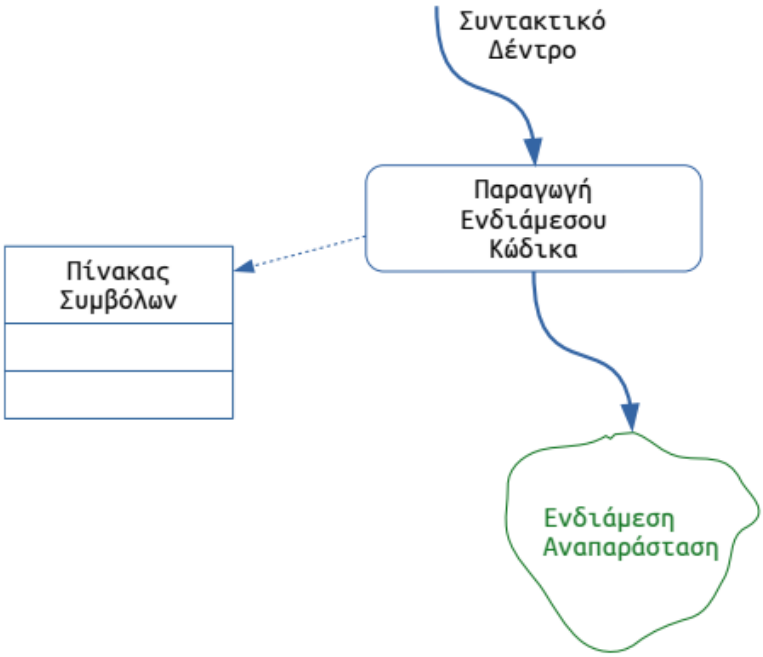
- **add_oper()**: Αναγνωρίζει και επιστρέφει πρόσθεση ή αφαίρεση (+, -) ως τελεστές μεταξύ όρων.

- **mul_oper()**: Αναγνωρίζει και επιστρέφει πολλαπλασιασμό ή διαίρεση (*, /) ως τελεστές μεταξύ παραγόντων.

2.3 Ενδιάμεσος Κώδικας:



Ο ενδιάμεσος κώδικας (intermediate code) αποτελεί μια, χαμηλότερου επιπέδου αναπαράσταση του αρχικού προγράμματος, η οποία δημιουργείται από τον συντακτικό αναλυτή κατά τη φάση της μετάφρασης. Χρησιμοποιείται ως μεταβατικό στάδιο μεταξύ της ανάλυσης της σύνταξης και της παραγωγής του τελικού εκτελέσιμου κώδικα. Ο σκοπός του είναι να απλοποιήσει τη διαδικασία μετάβασης από μια υψηλού επιπέδου γλώσσα προγραμματισμού σε γλώσσα μηχανης (assembly), επιτρέποντας παράλληλα ευκολότερη βελτιστοποίηση του προγράμματος. Στο πλαίσιο αυτού του μεταγλωττιστή, ο ενδιάμεσος κώδικας παίρνει τη μορφή ενός συνόλου τετράδων (quads), καθεμία από τις οποίες εκφράζει μια απλή εντολή με τη δομή [τελεστής, τελούμενο1, τελούμενο2, τελούμενο3].



Κάθε τετράδα χαρακτηρίζεται από έναν μοναδικό αριθμό που βρίσκεται μπροστά της. Όταν η εκτέλεση μίας τετράδας τελειώσει τότε εκτελείται η τετράδα με το αμέσως μεγαλύτερο αριθμό, εκτός αν η τετράδα που μόλις εκτελέστηκε υποδήξει κάτι διαφορετικό.

Για την υλοποίηση του ενδιάμεσου κώδικα δημιουργήσαμε την κλάση Quad στην οποία μέσα υλοποιούμε και όλες τις βοηθητικές μεθόδους που θα χρησιμοποιησουμε στο κομμάτι του Συντακτικού Αναλυτή. Η κλάση Quad αναπαριστά κάθε τετράδα του ενδιάμεσου κώδικα. Κάθε τετράδα περιλαμβάνει έναν μοναδικό αριθμό ταυτότητας ID, τον τελεστή op, δύο τελεστέους x και y, καθώς και το αποτέλεσμα της ενέργειας z (που είναι συνήθως μια νέα προσωρινή μεταβλητή ή ετικέτα).

<u>Ανάλυση βοηθητικών μεθόδων</u>:

- **genquad(op, x, y, z)**: Η genquad είναι υπεύθυνη για τη δημιουργία μιας νέας τετράδας και την προσθήκη της στη λίστα quadList. Αυξάνει τον μετρητή counter_nextquad κάθε φορά που δημιουργείται νέα τετράδα και αναθέτει αυτό το ID στην καινούρια τετράδα. Είναι μια από τις πιο βασικές συναρτήσεις του ενδιάμεσου κώδικα, καθώς από εδώ κατασκευάζονται όλες οι εντολές του προγράμματος σε μορφή τετράδων.

- **nextquad()**: Η nextquad επιστρέφει το ID της επόμενης τετράδας που πρόκειται να παραχθεί.

- **newtemp()**: Η newtemp δημιουργεί μια νέα προσωρινή μεταβλητή, η οποία ονομάζεται με το πρόθεμα t@ ακολουθούμενο από έναν αριθμό. Αυτές οι μεταβλητές χρησιμοποιούνται για να αποθηκεύσουν ενδιάμεσα αποτελέσματα εκφράσεων.

- **backPatch(lst, z)**: Η backPatch διατρέχει μία λίστα που αποτελείται από δείκτες σε τετράδες των οποίων το τελευταίο τελούμενο δεν ειναι συμπληρωμένο. Επισκέπτεται μία μία τις τετράδες αυτές και τις συμπληρώνει με την ετικέτα z.

- **intermediateCodeGenerator()**: Η intermediateCodeGenerator γράφει σε αρχείο όλες τις τετράδες που έχουν παραχθεί, χρησιμοποιώντας τη μέθοδο toString(). Είναι η τελική φάση εξαγωγής του ενδιάμεσου κώδικα για σκοπούς καταγραφής ή επεξεργασίας από επόμενο στάδιο του compiler που είναι η παραγωγή του τελικού κώδικα.

Συναρτήσεις **emptyList()**, **makeList(x)**, **mergeList(list1, list2)**:

Αυτές οι συναρτήσεις διαχειρίζονται λίστες ετικετών (labels) που χρησιμοποιούνται για backpatching.

- **emptyList()**: απλώς επιστρέφει μια κενή λίστα.
- **makeList(x)**: δημιουργεί μια λίστα που περιέχει μόνο το στοιχείο x.
- **mergeList(list1, list2)**: ενώνει δύο λίστες και επιστρέφει το αποτέλεσμα.

<u>Ανάλυση δομών δεδομένων ενδιάμεσου κώδικα:</u>

- **Αρχή και Τέλος Block**

```
begin_block, name, _, _
end_block, name, _, _
```

Στην program_block() με την βοήθεια της genquad δημιουργούμε τετράδες που σηματοδοτούν την αρχη και το τέλος του ενδιάμεσου κώδικα που παρήχθη για μία συνάρτηση, διαδικασία ή για το κυρίως πρόγραμμα. Με αυτόν τον τρόπο στην αρχή του κώδικα μίας συνάρτησης, διαδικασίας η της main, τοποθετούμε μια begin_block με το αντίστοιχο όνομα και στο τέλος μια end_block.

- **Εκχώριση**

```
:= , source, _, target
```

Στην assignment_stat() δημιουργούμε μία τετράδα η οποία σηματοδοτεί την εκχώρηση της τιμής του source στο target

- **Αριθμητική Πράξη**

```
op, operand1, operand2, target
```

Εδώ γίνεται η αναπαράσταση όλων των πράξεων που υποστηρίζει η γλώσσα(πρόσθεση αφαίρεση, πολλαπλασιασμός, διαίρεση).

- <u>Πρόσθεση-Αφαίρεση</u>:
  Για να διαχειρίστουμε τις πράξεις της πρόσθεσης και τις αφαίρεσης κάνουμε χρήση των προσωρινών μεταβλητών έτσι ώστε το αποτέλεσμα να υπολογίζεται βηματικά. Για την συγκεκριμένη υλοποίηση προσθέτουμε στην expression() την ακόλουθη δομή.

  $$E \rightarrow T^1 \; (+ T^2 \; \{P_1\})^* \;\; \{P_2\}$$

  Όπου **{P1}**, φτιάχνουμε μία προσωρινη μεταβλητή w, με την βοήθεια της newtemp η οποία θα κρατήσει το μέχρι στιγμής αποτέλεσμα. Έπειτα παράγουμε μία τετράδα μέσω της οποίας προσθέτουμε το αποτέλεσμα στην t2 και τέλος τοποθετούμε συνολικό αποτέλεσμα στην t1.

```
w = newtemp()
genquad(operator1, t1, t2, w)
t1 = w
```

  Όπου **{P2}**, εάν δεν υπάρχει άλλο t2 τοποθετούμε το αποτέλεσμα στο t1.

  Σημείωση: Γίνεται έλεγχος μέσω της optional_sign() για το πρόσημο της πράξης.

- <u>Πολλαπλασιασμός-Διαίρεση</u>:
  Ακολουθούμε αντίστοιχα την ίδια λογικη προσθέτοντας στη term() την ακόλουθη δομή.

  $$T \rightarrow F^1 \; (\times F^2 \; \{P_1\})^* \;\; \{P_2\}$$

- **Λογικές Παραστάσεις – OR**
  Για την υλοποιήση των λογικών παραστάσεων και συγκεκριμενά του OR, προσθέσαμε στην condition() την παρακάτω δομή

  $B \rightarrow Q^1 \{P_1\} (or \{P_2\} Q^2 \{P_3\})^*$

  Όπου **{P1}**, μεταφέρουμε τις τετράδες απο την λίστα Q1 στην λίστα B.

  Όπου **{P2}**, συμπληρώνουμε όσες τετράδες μπορούν να συμπληρωθούν μέσα στον κανόνα.

  Όπου **{P3}**, συσσωρεύουμε, με την βοήθεια της mergeList, στη λίστα true τις τετράδες που δεν μπορούν να συμπληρωθούν και αντιστοιχούν σε αληθή αποτίμηση της λογικήε παράστασης. Η false λίστα περιέχει τη τετράδα η οποία αντιστοιχεί στη μη αληθή αποτίμηση της λογικής παράστασης.

- **Λογικές Παραστάσεις – AND**
  Για την υλοποιήση των λογικών παραστάσεων και συγκεκριμενά του AND, προσθέσαμε στην condition() την παρακάτω δομή.

  $Q \rightarrow R^1 \{P_1\} (and \{P_2\} R^2 \{P_3\})^*$

  Όπου **{P1}**, μεταφέρουμε τις τετράδες απο την λίστα R1 στην λίστα Q.

  Όπου **{P2}**, συμπληρώνουμε όσες τετράδες μπορούν να συμπληρωθούν μέσα στον κανόνα

  Όπου **{P3}**, συσσωρεύουμε, με την βοήθεια της mergeList, στη λίστα false τις τετράδες που δεν μπορούν να συμπληρωθούν και αντιστοιχούν σε μη αληθή αποτίμηση της λογικής παράστασης. Η true λίστα περιέχει τη τετράδα η οποία αντιστοιχεί στην αληθή αποτίμηση της λογικής παράστασης.

- **Λογικές Παραστάσεις**
  Στην boolfactor() προσθέσαμε τις τρεις ακόλουθες δομές για κάθε σενάριο των λογικών παραστάσεων.

  Μη ύπαρξη not:

  $R \rightarrow (B) \{P_1\}$

  Όπου **{P1}**, μεταφέρουμε τις τετράδες από την λίστα B στη λίστα R.

  Υπαρξη not:

  $R \rightarrow not (B) \{P_1\}$

  Όπου **{P1}**, αντιστρέφουμε και μεταφέρουμε τετράδες από την λίστα B στην λίστα R.

  Σχεσιακές συγκρίσεις:

  $R \rightarrow E^1 relop E^2 \{P_1\}$

  Όπου **{P1}**, δημιουργούμε μη συμπλωρεμένες τετράδες γαι την αληθή και μη αληθή αποτίμηση της σχεσιακης σύγκρισης.

- **Κλήση Υποπρογραμμάτων**

    - Κλήση διαδικασίας:
      Για κάθε κλήση διαδικασίας δημιουργούμε στην call_stat(), με την βοήθεια της genquad, την τετραδα genquad("call", id, "_", "_")

    - Κλήση συνάρτησης:
      Για κάθε κλήση συνάρτησης δημιουργούμε στην call_func(), μια προσωρινη μεταβλητή και δύο τετράδες της μορφής genquad ("par", temp, "RET", "_") , genquad("call", id, "_", "_"). Η μια είναι υπεύθηνη για την επιστορφή της συνάρτησης και η άλλη για την κλήση της.

- **Δομη while**
  Για την αναπαράσταση του ενδιάμεσου κώδικα, στη while_stat() προσθεσαμε τη δομη:
  S -> while {P1} B do {P2} S^1 {P3}

  Όπου **{P1}**, κάνουμε Bquad = nextquad().

  Όπου **{P2}**, συμπληρώνουμε τις τετράδες που έχουν ασυμπλήρωτες και γνωρίζουμε πλέον ότι πρέπει να συμπληρωθούν με την επόμενη τετράδα.

  Όπου **{P3}**, μεταβαίνουμε στην αρχή της συνθήκης ώστε να ξαναγίνει έλεγχος.

- **Δομή if**
  Για την αναπαράσταση του ενδιάμεσου κώδικα, στη if_stat() προσθεσαμε τη δομη:
  S -> if B then {P1} S^1 {P2} TAIL {P3}

  Όπου **{P1}**, συμπληρώνουμε τις τετράδες στο if που έχουν μείνει ασυμπλήρωτες και γνωρίζουμε τώρα ότι πρέπει να συμπλήθούν με την επόμενη τετράδα.

  Όπου **{P2}**, συμπληρώνουμε τις τετράδες στο else που έχουν μείνει ασυμπλήρωτες και γνωρίζουμε τώρα ότι πρέπει να συμπλήθούν με την επόμενη τετράδα.

  Όπου **{P3}**, εξασφαλίζουμε ότι εάν εκτελεστούν οι εντολές του if δεν θα εκτελεστούν οι εντολές του else.

- **Είσοδος**
  Για να αναπαραστήσουμε την είσοδο σε ενδιάμεσο κώδικα, προσθέτουμε στην input_stat() την δομη:
  S -> input (id) {P1}

  Όπου **{P1}**, δημιουργουμε την τετράδα genquad("inp",id, "_", "_").

- **Έξοδος**
  Για να αναπαραστήσουμε την έξοδο σε ενδιάμεσο κώδικα, προσθέτουμε στην print_stat() την δομη:
  S -> print (E) {P2}

  Όπου **{P2}**, δημιουργουμε την τετράδα genquad("out",exp,"_","_").

- **Δομή For**
  Για την αναπαράσταση του ενδιάμεσου κώδικα στη for_stat() προσθέσαμε τη δομή:

  S -> for id:= E1 {P1} to E2 {P2} do {P3} <step> {P4}

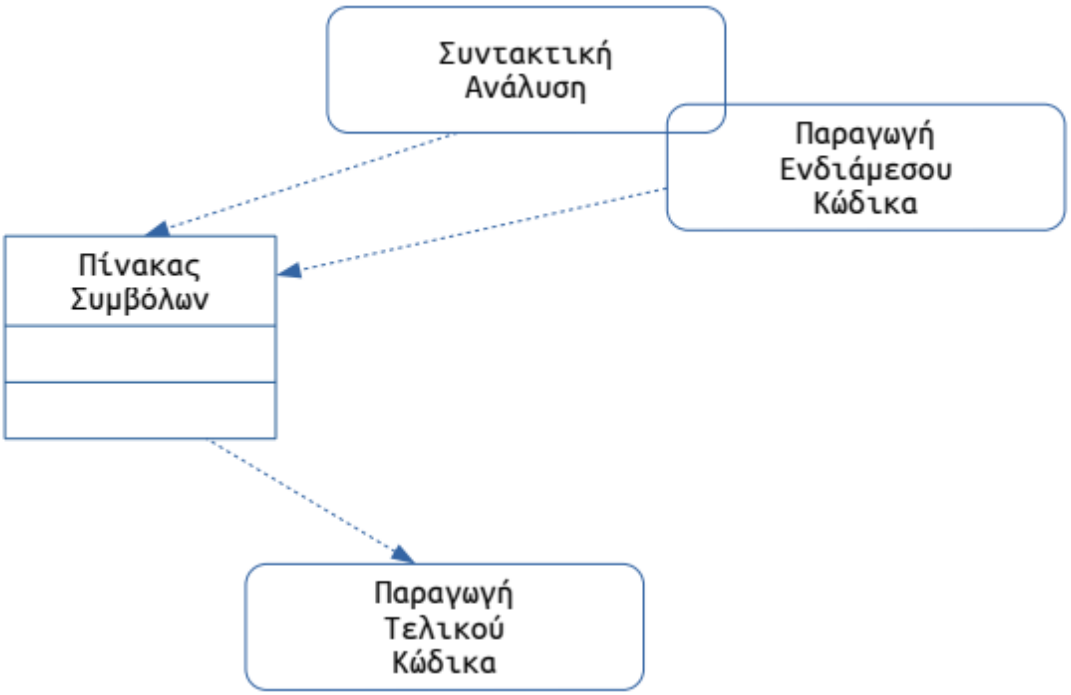
  Όπου **{P1}**, πραγματοποιείται η αρχικοποίηση του μετρητή του βρόχου, όπου δημιουργείται τετράδα τύπου genquad(":=", exp_start, "_", id) για την ανάθεση της αρχικής τιμής E1 στη μεταβλητή id

  Οπού **{P2},** υπολογίζεται η τελική τιμή E2 και εξετάζεται αν έχει δοθεί βήμα (με_βήμα). Αν δεν έχει καθοριστεί, θεωρείται εξ ορισμού ίσο με 1. Ανάλογα με την τιμή του βήματος step, δημιουργείται και η κατάλληλη συνθήκη: αν το βήμα είναι θετικό ή προσωρινή μεταβλητή, τότε η συνθήκη επανάληψης είναι id <= E2 και αποτυπώνεται με την τετράδα genquad("<= ", id, exp_end, "_"). Αντίθετα, αν το βήμα είναι αρνητικός αριθμός, τότε η συνθήκη είναι id >= E2 και αποτυπώνεται με την τετράδα genquad(">=", id, exp_end, "_"). Στη συνέχεια δημιουργείται τετράδα άλματος (genquad("jump", "_", "_", "_")) που οδηγεί προσωρινά εκτός βρόχου και η θέση της αποθηκεύεται για μελλοντικό backpatching.

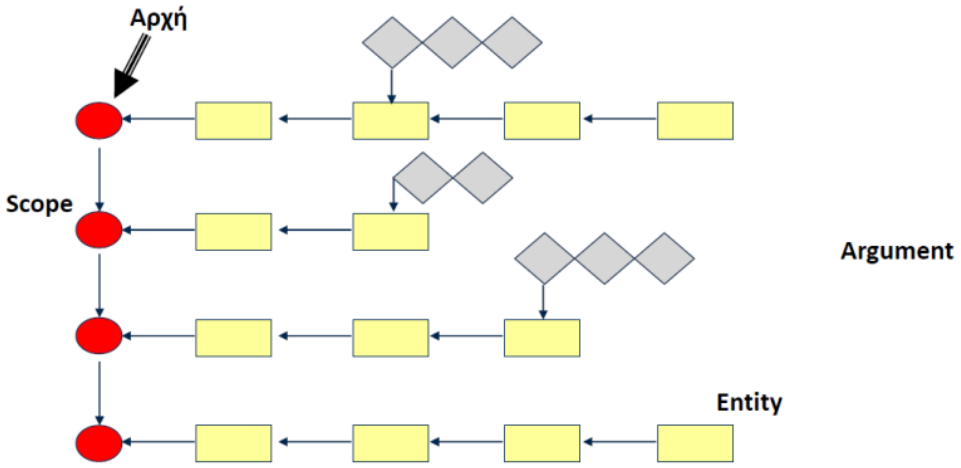
  Οπού **{P3},** καθορίζεται η μεταβολή του μετρητή. Αν το step είναι θετικός αριθμός ή προσωρινή μεταβλητή, τότε δημιουργείται η τετράδα genquad("+", id, step, id) για την αύξηση του μετρητή. Αντίστοιχα, αν είναι αρνητικός αριθμός, τότε δημιουργείται η τετράδα genquad("-", id, abs(step), id), όπου abs(step) είναι η απόλυτη τιμή του βήματος.

  Οπού **{P4}**, μετά την ανάλυση του σώματος του βρόχου μέσω της sequence(), δημιουργείται τετράδα jump για επιστροφή στη συνθήκη ελέγχου του βρόχου. Τέλος, εφαρμόζεται η backPatch(false, nextquad()) ώστε να ενημερωθεί η τετράδα εξόδου από τον βρόχο.

2.4 Πίνακας Συμβόλων:



Ο πίνακας συμβόλων αποτελεί ένα από τα πιο θεμελιώδη δομικά στοιχεία ενός μεταγλωττιστή, καθώς χρησιμοποιείται για την αποθήκευση και διαχείριση όλων των σημασιολογικών πληροφοριών που σχετίζονται με τις μεταβλητές, τις συναρτήσεις και τις διαδικασίες ενός προγράμματος. Σε κάθε φάση της μεταγλώττισης όπου αναλύονται δηλώσεις, ο πίνακας συμβόλων προσφέρει έναν μηχανισμό οργάνωσης και αναζήτησης, επιτρέποντας την ορθή αντιστοίχιση μεταξύ ονομάτων και ιδιοτήτων τους, όπως τύπος, πεδίο (scope), offset στη μνήμη, είδος οντότητας (μεταβλητή, συνάρτηση κ.λπ.) και παραμέτρους. Η σωστή σχεδίαση και υλοποίηση του πίνακα αυτού είναι καθοριστικής σημασίας για τη σημασιολογική ανάλυση και τη δημιουργία ορθού ενδιάμεσου και τελικού κώδικα. Ο πίνακας συμβόλων που υλοποιήσαμε χαρακτηρίζεται από την ακόλουθη εικόνα.



Για την υλοποίηση του πίνακα συμβόλων δημιουργήσαμε τις ακόλουθες κλάσεις και τις αντίστοιχες βοηθητικές συναρτήσεις:

- **Κλάση Entity**
  Αντιπροσωπεύει μία οντότητα στον πίνακα συμβόλων (μεταβλητή, συνάρτηση, διαδικασία, παράμετρος) και έχει τα εξής βασικά χαρακτηριστικά.

  - name: το όνομα της οντότητας.
  - type: ο τύπος της (π.χ. variable, function, parameter).
  - offset: απόσταση απο την αρχή του εγγραφήματος δραστηριοποίησης.
  - startQuad: ετικέτα της πρώτης τετράδας του κώδικα της συνάρτησης.
  - frameLength: συνολικό μήκος εγγραφήματος δραστηριοποίησης.
  - argumentList: λίστα παραμέτρων.
  - value, parMode: επιπλέον πληροφορίες όπως τιμή και τρόπος περάσματος παραμέτρου.

- **Κλάση Scope**
  Αντιπροσωπεύει ένα επίπεδο που αφορά μία διακασία ή συνάρτηση, το οποίο κρατά τις αντίστοιχες οντότητες και παραμέτρους. Έχει τα εξής χαρακτηριστικά.

  - entityList: η λίστα των οντοτήτων (τύπου Entity) που δηλώθηκαν στο scope.
  - nestingLevel: το βάθος φωλιάσματος του scope (0 για το κύριο πρόγραμμα).

  Επιπλέον δημιουργήσαμε κάποιες βοηθητικες μεθόδους.

  - appendEntity(entity): προσθέτει οντότητα στη λίστα.
  - getScopeOffset(): επιστρέφει το offset της τελευταίας έγκυρης οντότητας του scope στο οποίο βρισκόμαστε.
  - getEntityNames(): επιστρέφει λίστα με τα ονόματα των οντοτήτων στο scope.

- **Κλάση Argument**
  Αναπαριστά μία **παράμετρο** σε κλήση συνάρτησης/διαδικασίας και έχει τα εξής χαρακτηριστικά.

  - parMode: ο τρόπος περάσματος (με τιμή, με αναφορά).

Με βάση τις παραπάνω συναρτήσεις που υλοποιήσαμε στις εκάστοτε κλάσεις, θα πάμε και θα τις χρησιμοποιήσουμε κατάλληλα στις συναρτήσεις του συντακτικού αναλυτή.

- <u>Ενσωμάτωση του Πίνακα Συμβόλων στη **program()**</u>
  Αρχικά, δημιουργούμε ένα νέο scope με βάση το τρέχον nesting level. Αυτό το scope αντιστοιχεί στο κύριο πρόγραμμα και αντιπροσωπεύει τον χώρο ορατότητας των μεταβλητών, συναρτήσεων και διαδικασιών που θα δηλωθούν εντός του. Έπειτα, δημιουργείται μία νέα οντότητα τύπου prog, που αναπαριστά το ίδιο το πρόγραμμα, με όνομα το αναγνωριστικό που διαβάστηκε και offset 8 (τυπικά το offset εκκίνησης για το εγγράφημα δραστηριοποίησης του κύριου προγράμματος). Στη συνέχεια προσθέτουμε την οντότητα στο scope και αντίστοιχα το scope στη στοίβα scopeList. Μετά το πέρας της επεξεργασίας του μπλοκ του προγράμματος, το scope αφαιρείται από τη στοίβα και μεταφέρεται στη λίστα deletedScopes, η οποία περιέχει όλα τα scopes που έχουν επεξεργαστεί πλήρως και είναι έτοιμα να χρησιμοποιηθούν από την έξοδο του πίνακα συμβόλων.

- Ενσωμάτωση του Πίνακα Συμβόλων στη **varlist()**
  Αρχικά, δημιουργούμε μία νέα οντότητα τύπου var με ονομα το id της μεταβλητής και offset υπολογισμένο με βάση το offset της τελευταίας οντότητας προσθέτοντας 4 bytes. Η νέα οντότητα προστίθεται στο entityList του scope μόνο αν το όνομα της μεταβλητής δεν έχει ήδη δηλωθεί στο ίδιο scope, αποτρέποντας έτσι διπλές δηλώσεις.

- Ενσωμάτωση του Πίνακα Συμβόλων στη **func()**
  Αρχικά, προσθέτουμε την νέα συνάρτηση στο τρέχον scope(δεν μας ενδιαφέρει το offset). Έπειτα αυξάνουμε το επίπεδο εμφώλευσης (nesting) και δημιουργείται νέο scope για να περιλαμβάνει το σώμα της συνάρτησης και τις τοπικές της μεταβλητές. Με την ολοκλήρωση της ανάλυσης της συνάρτησης, υπολογιζουμε το offset της συνάρτησης στο εγγράφημα δραστηριοποίησης, αφαιρούμε το scope απο την στοίβα και το αποθηκευουμε στην λίστα deletedScopes για την αναπαράσταση στον πίνακα συμβόλων.

- Ενσωμάτωση του Πίνακα Συμβόλων στη **proc()**
  Ακολουθούμε την ίδια ακριβώς διαδικασία που ακολουθήσαμε και για την υλοποίηση της **func(),** με την μόνη διαφορά ότι η οντότητα που αποθηκεύουμε ειναι τύπου proc.

- Ενσωμάτωση του Πίνακα Συμβόλων στη **func_input()**
  Αρχικα, παίρνουμε το scope της συνάρτησης/διαδικασίας που αυτή τη στιγμή ορίζουμε και βρίσκουμε την τελευταία οντότητα που προστέθηκε. Έπειτα, δημιουργούμε νέο αντικείμενο Argument, με τιμή "in" και το προσθέτουμε στη λίστα των ορισμάτων της οντότητας συνάρτηση/διαδικασία.Τέλος, ορίζουμε την τετράδα από την οποία ξεκινά το σώμα της συνάρτησης/διαδικασίας.

- Ενσωμάτωση του Πίνακα Συμβόλων στη **func_output()**
  Ακολουθούμε την ίδια διαδικασία όπως και στη **func_input(),** με την διαφορά ότι εδώ δημιουργούμε ένα νέο αντικείμενο Argument, με τιμή "io"

- Ενσωμάτωση του Πίνακα Συμβόλων στη **assignment_stat()**
  Αρχικά, πηγαίνουμε στο τρέχον scope βάσει του nesting και δημιουργούμε μία νέα οντότητα τυπου temp καθώς στο πίνακα συμβόλων είναι απαραίτητο να αποθηκεύσουμε και τις προσωρινές μεταβλητές. Αν η οντότητα δεν είναι κάποιος και είναι οντως προσωρινή μεταβλητή, την προσθετουμε στο scope.

2.5 Σημασιολογική Ανάλυση:

Κατά τη φάση της σημασιολογικής ανάλυσης, το κύριο ζητούμενο είναι να διασφαλιστεί ότι ένα πρόγραμμα, αν και μπορεί να είναι συντακτικά ορθό, ακολουθεί και τους κανόνες που επιβάλλει η σημασιολογία της γλώσσας. Με τη βοήθεια του πίνακα συμβόλων, πραγματοποιούνται έλεγχοι που δεν μπορούν να εκφραστούν μόνο μέσω της γραμματικής, όπως η ύπαρξη και η ορθή δήλωση μεταβλητών, η συμβατότητα τύπων, αλλά και η σωστή χρήση παραμέτρων σε κλήσεις συναρτήσεων ή διαδικασιών. Επιπλέον, ελέγχεται αν οι εντολές return τοποθετούνται σωστά, αν μια συνάρτηση επιστρέφει τιμή όταν πρέπει, καθώς και αν οι παραστάσεις υπακούουν στους κανόνες τύπων. Όλοι αυτοί οι έλεγχοι διασφαλίζουν ότι το πρόγραμμα δεν έχει εσωτερικές αντιφάσεις ή παραλείψεις που θα οδηγούσαν σε σφάλματα κατά την εκτέλεση ή μετάφραση του κώδικα.

Για την υποστήριξη της σημασιολογικής ανάλυσης κατά τη μετάφραση ενός προγράμματος, υλοποιήθηκαν βοηθητικές συναρτήσεις οι οποίες διευκολύνουν την αναζήτηση οντοτήτων στον πίνακα συμβόλων, την επιβεβαίωση ύπαρξης μεταβλητών, και τον έλεγχο συμβατότητας των παραμέτρων κατά την κλήση συναρτήσεων ή διαδικασιών

- **getEntityFromScopeList2(entityName)**
  Παρόμοια με την προηγούμενη, αλλά αναζητά την οντότητα στο τρέχον ενεργό scope (scopeList), δηλαδή σε ενεργές περιοχές του προγράμματος. Την χρησιμοποιούμε όταν θέλουμε να βρούμε μια μεταβλητή ή συνάρτηση που πρέπει να είναι προσβάσιμη στο σημείο που βρίσκεται ο συνατκτικός αναλυτής.

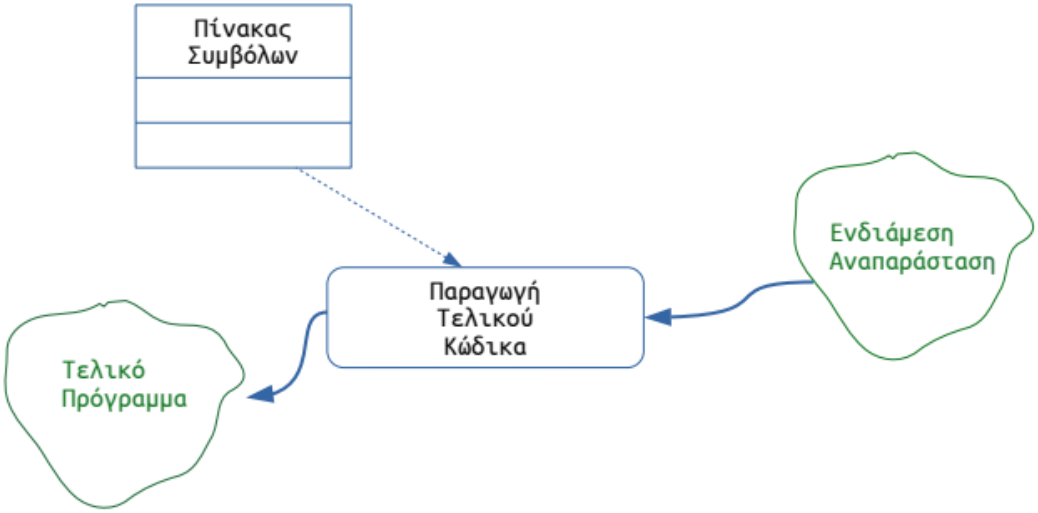
- **resolveVariable(variable)**
  Επιστρέφει True αν μια μεταβλητή με το δοσμένο όνομα υπάρχει σε κάποιο από τα ενεργά scopes. Χρησιμοποιείται για να εντοπίζεται εάν μια μεταβλητή έχει δηλωθεί πριν τη χρήση της.

- **getFunctionVarialbles(function)**
  Επιστρέφει τον αριθμό των παραμέτρων της συνάρτησης (ή διαδικασίας). Έπειτα αναζητεί την αντίστοιχη οντότητα στο τελευταίο scope που εχει δημιουργηθεί (αν δεν βρεθεί επιστρέφουμε -1). Ο συγκεκριμένος ελεγχος είναι απαραίτητος για να επιβαιβεώσουμε ότι ο αριθμός παραμέτρων που περνάμε σε μια κλήση είναι ο αναμενόμενος.

- checkFunctionVariables(function, variables)
  Πραγματοποιεί έλεγχο ορθότητας μεταξύ των παραμέτρων(με τιμή/με αναφορά). Αρχικά, ανακτά τις παραμέτρους που είναι περασμένες είτε με τιμή (in), είτε με αναφορά (io) από τον πίνακα συμβόλων της συνάρτησης. Έπειτα ελέγχουμε αν ο αριθμός και ο τρόπος περάσματος των παραμέτρων συμφωνεί με τη δήλωση. Αν όλοι οι τύποι παραμέτρων έχουν ικανοποιηθεί, η συνάρτηση επιστρέφει True, αλλιώς False.

Συνολικά, οι παραπάνω συναρτήσεις επιτρέπουν την εφαρμογή βασικών σημασιολογικών ελέγχων όπως η ορθή χρήση μεταβλητών και παραμέτρων, η αντιστοίχιση παραμέτρων κατά την κλήση συναρτήσεων/διαδικασιών και η οργάνωση της πληροφορίας των scopes μέσω του πίνακα συμβόλων.

2.6 Παραγωγή Τελικού Κώδικα:



Η παραγωγή τελικού κώδικα αποτελεί το τελευταίο στάδιο της μεταγλωττιστικής διαδικασίας, όπου το ενδιάμεσο πρόγραμμα μετατρέπεται σε μια πιο «πραγματική» μορφή. Σε αυτή τη φάση, οι αφηρημένες εντολές που έχουν παραχθεί από τον ενδιάμεσο κώδικα μετατρέπονται σε πιο συγκεκριμένες οδηγίες, με σαφή χρήση καταχωρητών, διευθύνσεων και οργάνωση μνήμης. Η διαδικασία αυτή πρέπει να διατηρεί την ορθότητα και το νόημα του αρχικού προγράμματος, ενώ ταυτόχρονα επιδιώκει την αποδοτική διαχείριση πόρων.

Για την παραγωγή ενδιάμεσου κώδικα υλοποιήσαμε τις ακόλουθες συναρτήσεις:

- **gnlvCode(n)**
  Η gnlvCode(n) ηταν η πρώτη βασική συνάρτηση που υλοποιήσαμε στο κομμάτι του σταδίου παραγωγής τελικού κώδικα, η οποία αφορά τη δημιουργία των εντολών για την προσπέλαση μη τοπικών μεταβλητών. Πιο συγκεκριμένα υλοποιεί έναν μηχανισμό, τέτοιο ώστε να μπορεί μία συνάρτηση ή διαδικασία να προσπελάσει μεταβλητές που έχουν δηλωθεί σε εξωτερικά scopes. Σκοπός της είναι να δημιουργήσει και να επιστρέψει μία λίστα εντολών ενδιάμεσου κώδικα (τύπου MIPS) που επιτρέπει την εύρεση της διεύθυνσης μιας τοπικής μεταβλητής n. Η μεταβλητή αυτή δεν ανήκει στο ενεργό scope που βρισκόμαστε, αλλά σε κάποιον πρόγονό του.

  Για την παραγωγή τελικού κώδικα, η εύρεση μιας μεταβλητής που δεν είναι δηλωμένη στο current scope απαιτεί χρήση του **stack pointer (sp)** για την προσπέλαση ανωτέρων περιοχών**.** Αρχικά, φορτώνεται στον καταχωρητή **t0** η τιμή που βρίσκεται στη θέση **-4(sp)**, η οποία περιέχει τη διεύθυνση του ενεργού record της αμέσως εξωτερικής συνάρτησης. Καθώς ανεβαίνουμε ιεραρχικά στα nesting levels, πραγματοποιούνται διαδοχικά **lw t0, -4(t0)** ώστε να φτάσουμε στο record όπου δηλώνεται η ζητούμενη μεταβλητή. Τέλος, με την εντολή addi **t0, t0, -offset**, υπολογίζεται η τελική διεύθυνση της μεταβλητής εντός του frame. Η διαδικασία αυτή καθιστά δυνατή την πρόσβαση σε μη τοπικές μεταβλητές.

- **loadvar(quadvar, tempregister)**
  Η συνάρτηση loadvar δημιουργεί κώδικα για τη φόρτωση της τιμής μιας μεταβλητής ή μιας αριθμητικής σταθεράς σε έναν καταχωρητή προσωρινής χρήσης. Πρώτα, γίνεται αναζήτηση της μεταβλητής σε όλα τα scopes που έχουν αφαιρεθεί μέσω της λίστας deletedScopes, προκειμένου να εντοπιστεί το nesting level στο οποίο δηλώθηκε η μεταβλητή.
  Ανάλογα με τον τύπο της μεταβλητής, το scope της και τον τρόπο περάσματος της (με τιμή ή με αναφορά), ο αντίστοιχος παραγόμενος κώδικας διαφέρει:

  - **Αριθμητικές σταθερές**: Εάν η μεταβλητή είναι ακέραιος αριθμός (digit), παράγεται εντολή li για άμεση φόρτωση της τιμής.

  - **Τοπικές μεταβλητές στη main**: Εάν η μεταβλητή βρίσκεται στο nesting level 0, τότε η τιμή της φορτώνεται από τον πίνακα με τα ενεργά scopes του κυρίως προγράμματος με βάση το **s0**.

  - **Τοπικές μεταβλητές ή παράμετροι με τιμή στο ίδιο nesting level**: Χρησιμοποιείται το **sp** για άμεση πρόσβαση.

  - **Παράμετροι με αναφορά στο ίδιο nesting level**: Πρώτα φορτώνεται η διεύθυνση της μεταβλητής και μετά το περιεχόμενο της διεύθυνσης με χρήση δύο εντολών **lw**.

  - **Μεταβλητές από scope πρόγονο(πέρασμα με αναφορά)**: Χρησιμοποιείται η συνάρτηση gnlvCode για εύρεση της διεύθυνσης της μεταβλητής στο stack και στη συνέχεια μια εντολή lw για τη φόρτωση της τιμής.

  - **Παράμετροι με αναφορά από scope πρόγονο**: Εφαρμόζεται gnlvCode για εύρεση της διεύθυνσης του δείκτη, έπειτα γίνεται **lw** για να ανακτηθεί η διεύθυνση της μεταβλητής, και τελικά **lw** για να φορτωθεί η τιμή.

Αν κανένας από τους παραπάνω κανόνες δεν ταιριάζει, εμφανίζεται μήνυμα σφάλματος και επιστρέφεται κατάλληλη ειδοποίηση.

- **storevar(r, v)**
  Η συνάρτηση storevar παράγει τον κατάλληλο τελικό ή τελικό κώδικα για την αποθήκευση της τιμής που περιέχει ο καταχωρητής **t{r}** σε μια μεταβλητή v. Η διαδικασία περιλαμβάνει την εύρεση της οντότητας που αντιστοιχεί στη μεταβλητή v, μέσα από τα deleted scopes ώστε να προσδιοριστεί το nesting level στο οποίο δηλώθηκε.
  Ανάλογα με την μεταβλητή(τοπική, προσωρινή, καθολική ή παράμετρος) το nesting level και το τρόπο περάσματος (με τιμή ή αναφορά), δημιουργούνται διαφορετικές εντολές:

  - **Καθολικές μεταβλητές (στη main)**: Η αποθήκευση γίνεται μέσω του καταχωρητή βάσης του κυρίως προγράμματος **s0**

  - **Τοπικές μεταβλητές, παράμετροι με τιμή, προσωρινές μεταβλητές στο ίδιο scope**: Η αποθήκευση γίνεται απευθείας μέσω του **sp** και offset από τον πίνακα ενεργών εγγραφών.

- **Παράμετροι με αναφορά στο ίδιο scope**: Πρώτα ανακτάται η διεύθυνση της μεταβλητής με **lw**, και κατόπιν η τιμή αποθηκεύεται σε αυτήν με **sw**.

- **Μεταβλητές ή παράμετροι με τιμή από προγονικό scope**: Χρησιμοποιούμε τη gnlvCode για τον υπολογισμό της καθολικής διεύθυνσης και στη συνέχεια την αποθηκεύουμε.

- **Παράμετροι με αναφορά από προγονικό scope**: Αρχικά με χρήση της gnlvCode βρίσκουμε την διεύθυνση, στη συνέχεια ανακτούμε τον δείκτη τον δείκτη με **lw** και τέλος αποθηκεύουμε την τιμή στην διεύθυνση.

Σε περίπτωση που καμία από τις παραπάνω περιπτώσεις δεν πληρείται , εμφανιζουμε τα αντίστοιχα μηνύματα σφάλματος.

- **quadToFinalCode(quad)**
  Η συνάρτηση είναι υπεύθυνη για τον έλεγχο του τύπου της εντολής μέσω του πεδίου quad.op. Ανάλογα με την τιμή του, εκτελείται η αντίστοιχη μετάφραση:

  - **Άλματα και συγκρίσεις**: Για εντολές όπως **jump, =, <, <=, <>, >, >=**, η συνάρτηση δημιουργεί τις αντίστοιχες εντολές RISCV (j, beq, blt, ble, bne, bgt, bge). Πριν από την εντολή άλματος, οι τιμές των μεταβλητών φορτώνονται σε προσωρινούς καταχωρητές μέσω της loadvar.

  - **Αναθέσεις και αριθμητικές πράξεις**: Η εντολή **:=** μεταφράζεται σε φόρτωση της τιμής και αποθήκευσή της στη σωστή θέση μνήμης με τη βοήθεια της storevar. Οι αριθμητικές πράξεις (+, -, *, /) μεταφράζονται σε αντίστοιχες εντολές RISCV (add, sub, mul, div), με τις τιμές να φορτώνονται σε καταχωρητές και το αποτέλεσμα να αποθηκεύεται κατάλληλα.

  - **Επιστροφή τιμής**: Η εντολή retv χειρίζεται την επιστροφή τιμής από συνάρτηση. Η τιμή φορτώνεται και αποθηκεύεται στη θέση που δείχνει ο δείκτης επιστροφής.

  - **Παράμετροι και κλήσεις συναρτήσεων**: Η εντολή par διαχειρίζεται τις παραμέτρους συναρτήσεων, λαμβάνοντας υπόψη τον τρόπο περάσματος (κατά τιμή ή κατά αναφορά). Η συνάρτηση εξετάζει το επίπεδο φωλεοποίησης και χρησιμοποιεί τις loadvar, storevar και gnlvCode για τη σωστή διαχείριση των παραμέτρων. Η εντολή call χειρίζεται την κλήση συναρτήσεων, ρυθμίζοντας το πλαίσιο ενεργοποίησης και χρησιμοποιώντας τις εντολές jal και jr για τη μετάβαση και επιστροφή από τη συνάρτηση.

  - **Διαχείριση μπλοκ**: Οι εντολές begin_block και end_block σηματοδοτούν την αρχή και το τέλος ενός μπλοκ κώδικα. Η συνάρτηση χειρίζεται την αποθήκευση και αποκατάσταση του καταχωρητή επιστροφής (ra) και τη ρύθμιση του δείκτη στοίβας.

  - **Είσοδος/Εξοδος**: Οι εντολές inp και out διαχειρίζονται την είσοδο και έξοδο δεδομένων, χρησιμοποιώντας τις εντολές MIPS li, ecall και move για την αλληλεπίδραση με το χρήστη.

  - Τέλος υλοποιήσαμε **test_quadToFinalCode_and_write_output** η οποία έχει ως σκοπό τη δοκιμή της διαδικασίας μετατροπής των quads σε τελικό κώδικα RISCV και την καταγραφή του παραγόμενου κώδικα σε αρχείο.

Κεφάλαιο 3ο: Παραδείγματα ορθής λειτουργείας

3.1demo1

greek++

```
πρόγραμμα simple_demo
  δήλωση i, a, b, c, result

  συνάρτηση diplasio(x)
  διαπροσωπεία
  είσοδος x


  αρχή_συνάρτησης
    diplasio := x * 2
  τέλος_συνάρτησης


  αρχή_προγράμματος
    a := 3;
    b := diplasio(a);
    γράψε b;

    για i := 1 έως 5 επανάλαβε
      γράψε i
    για_τέλος
  τέλος_προγράμματος
```

Το πρόγραμμα simple_demo αποτελεί ένα απλό παράδειγμα υλοποίησης βασικών δομών της γλώσσας greek++ και περιλαμβάνει μεταβλητές, κλήση συνάρτησης, και βρόχο επανάληψης **για**. Αρχικά, ορίζεται η συνάρτηση diplasio(x) η οποία επιστρέφει το διπλάσιο της παραμέτρου x. Στο κύριο σώμα του προγράμματος, η μεταβλητή a λαμβάνει την τιμή 3 και στη συνέχεια καλείται η συνάρτηση diplasio(a) για να υπολογιστεί και να αποθηκευτεί στη μεταβλητή b το διπλάσιο του a. Το αποτέλεσμα (δηλαδή 6) εκτυπώνεται στην έξοδο. Ακολουθεί ένας βρόχος **για** από το 1 έως το 5, ο οποίος εκτυπώνει διαδοχικά τους αριθμούς από το 1 μέχρι το 5. Το πρόγραμμα αυτό επιδεικνύει τη χρήση παραμέτρων σε συναρτήσεις, την ανάθεση και την έξοδο τιμών, καθώς και τη χρήση της εντολής επανάληψης.

intermediate code

```
1: begin_block , diplasio , _ , _
2: * , x , 2 , t@1
3: retv , t@1 , _ , _
4: end_block , diplasio , _ , _
5: begin_block , simple_demo , _ , _
6: := , 3 , _ , a
7: par , a , CV , _
8: par , t@2 , RET , _
9: call , diplasio , _ , _
10: := , t@2 , _ , b
11: out , b , _ , _
12: := , 1 , _ , i
13: <= , i , 5 , 15
14: jump , _ , _ , 18
15: + , i , 1 , i
16: out , i , _ , _
17: jump , _ , _ , 13
18: halt , _ , _ , _
19: end_block , simple_demo , _ , _
```

Ο παρακάτω ενδιάμεσος κώδικας αναπαριστά τις βασικές εντολές του προγράμματος simple_demo σε μορφή τετράδων(quads), περιλαμβάνοντας την υλοποίηση της συνάρτησης diplasio, την κύρια ροή του προγράμματος, καθώς και τις αντίστοιχες εντολές για παράμετρο, κλήση και επανάληψη:

- Γραμμές 1–4: Περιγράφουν την υλοποίηση της συνάρτησης diplasio. Η εντολή begin_block σηματοδοτεί την έναρξη της συνάρτησης, ενώ η εντολή * υπολογίζει το γινόμενο της παραμέτρου x με το 2 και αποθηκεύει το αποτέλεσμα στην προσωρινή μεταβλητή t@1. Η εντολή retv χρησιμοποιείται για να επιστρέψει η τιμή της t@1 από τη συνάρτηση και το end_block σηματοδοτεί τη λήξη της.

- Γραμμές 5–19: Αφορούν το κύριο πρόγραμμα. Ξεκινά με begin_block και τελειώνει με end_block. Η εντολή := αναθέτει την τιμή 3 στη μεταβλητή a. Οι εντολές par δηλώνουν τις παραμέτρους που περνιούνται στη συνάρτηση: a ως παράμετρος τιμής (CV) και t@2 ως θέση επιστροφής (RET). Ακολουθεί η call της συνάρτησης diplasio και η ανάθεση του αποτελέσματος στη μεταβλητή b. Η τιμή του b εκτυπώνεται με την out.

- Γραμμές 12–17: Αντιστοιχούν στον βρόχο **για**. Αρχικά, γίνεται ανάθεση της τιμής 1 στη μεταβλητή i. Η σύγκριση <= ελέγχει αν i <= 5 και αν ναι, συνεχίζει στην επανάληψη, διαφορετικά μεταπηδά στο τέλος του βρόχου (γραμμή 18). Μέσα στο βρόχο αυξάνεται το i κατά 1 και εκτυπώνεται η τρέχουσα τιμή του. Τέλος, η εντολή jump επιστρέφει στον έλεγχο της συνθήκης.

- Γραμμή 18: Η εντολή halt σηματοδοτεί το τέλος του προγράμματος.
  Ο ενδιάμεσος αυτός κώδικας απεικονίζει με ακρίβεια τη ροή του αρχικού προγράμματος και τις λειτουργίες του, μεταφράζοντας τις δηλώσεις και εντολές σε μια ενδιάμεση μορφή κατάλληλη για μεταγενέστερα στάδια μετάφρασης, όπως βελτιστοποίηση ή δημιουργία τελικού κώδικα.

symbol table

```
Symbol Table:

Scope 1:
func : diplasio
x : Type: var , Offset: 12
t@1 : Type: temp , Offset: 16

Scope 0:
prog : simple_demo
i : Type: var , Offset: 12
a : Type: var , Offset: 16
b : Type: var , Offset: 20
c : Type: var , Offset: 24
result : Type: var , Offset: 28
diplasio : Type: func, FrameLength: 20,startQuad: 1 , in
t@2 : Type: temp , Offset: 32
```

Scope 1 (Εμβέλεια της συνάρτησης diplasio)

- **func : diplasio**
  Τύπος: func - Είναι η συνάρτηση diplasio.

- **x :**
  Τύπος: var - Είναι η παράμετρος εισόδου της συνάρτησης.
  Offset: 12 - Η μεταβλητή τοποθετείται στον offset 12 της στοίβας της συνάρτησης.

- **t@1 :**
  Τύπος: temp - Προσωρινή μεταβλητή που κρατά το αποτέλεσμα του x * 2.
  Offset: 16 - Καταλαμβάνει τον επόμενο διαθέσιμο χώρο μετά την παράμετρο.

Scope 0 (Κύριο πρόγραμμα simple_demo)

- **prog : simple_demo**
  Τύπος: prog - Είναι το κυρίως πρόγραμμα.

- **i, a, b, c, result :**
  Τύπος: var - Κανονικές μεταβλητές του κύριου προγράμματος.
  Offset: 12, 16, 20, 24, 28 - Τοποθετούνται διαδοχικά στη μνήμη, με σταθερή αύξηση offset.

- **diplasio :**
  Τύπος: func - Καταχώρηση για τη συνάρτηση diplasio.
  FrameLength: 20 - Η στοίβα της diplasio χρειάζεται 20 bytes συνολικά.
  startQuad: 1 - Ο τετραπλέος από τον οποίο ξεκινά η εκτέλεση της συνάρτησης στον ενδιάμεσο κώδικα.
  in – Σηματοδοτεί ότι η παράμετρος της συνάρτησης περνιέται με τιμή (CV)

- **t@2 :**
  Τύπος: temp - Προσωρινή μεταβλητή που χρησιμοποιείται για να κρατήσει την επιστρεφόμενη τιμή από τη diplasio.
  Offset: 32 - Ακολουθεί μετά από τις κανονικές μεταβλητές.

assembly

```
.data
   str_nl: .asciz "\n"
   .text

j Lmain
diplasio:
sw ra,(sp)
L_2:
lw t1,-12(sp)
li t2,2
mul t1,t1,t2
sw t1,-16(sp)
L_3:
lw t1,-16(sp)
lw t0,-8(sp)
sw t1,(t0)
L_4:
lw ra,(sp)
jr ra
Lmain:
addi sp,sp,-32
addi gp,sp,0
L_6:
li t1,3
sw t1,-16(sp)
L_7:
addi fp, sp,0
lw t1,-16(sp)
sw t1,-12(fp)
L_8:
addi t1,sp,-32
sw t1,-8(fp)
L_9:
sw sp,-4(fp)
addi sp,sp,0
jal diplasio
addi sp,sp,-0
L_10:
lw t1,-32(sp)
sw t1,-20(sp)
L_11:
li a7,1
lw t0,-20(sp)
addi a0,t0,0
ecall
la a0, str_nl
   li a7, 4
   ecall
L_12:
li t1,1
sw t1,-12(sp)
L_13:
lw t1,-12(sp)
li t2,5
ble t1,t2,L_15
L_14:
j L_18
L_15:
lw t1,-12(sp)
li t2,1
add t1,t1,t2
sw t1,-12(sp)
L_16:
li a7,1
lw t0,-12(sp)
addi a0,t0,0
ecall
  la a0, str_nl
    li a7, 4
    ecall
L_17:
j L_13
L_18:
li a7,10
ecall
  la a0, str_nl
    li a7, 4
    ecall
L_19:
lw ra,(sp)
jr ra
```

Αποτέλεσμα

```
6
2
3
4
5
6

-- program is finished running (0) --
```

3.2 demo2

greek++

```
πρόγραμμα full_demo
  δήλωση i, a, b, c, d, e, sum, res

  συνάρτηση athroisma(x, y)
  διαπροσωπεία
  είσοδος x, y

  αρχή_συνάρτησης
    athroisma := x + y
  τέλος_συνάρτησης

  αρχή_προγράμματος
    a := 2;
    b := 3;
    c := 1;
    d := 20;
    e := 0;

    εάν [ [a < b] και [c = d] ] ή [ [b > a] και [e = 0] ] τότε
      sum := athroisma(a, b)
    αλλιώς
      sum := athroisma(b, d)
    εάν_τέλος;

    γράψε sum;
    i := 0;
    όσο i < 3 επανάλαβε
      γράψε i;
      i := i + 1
    όσο_τέλος;

    για i := 1 έως 3 επανάλαβε
      εάν i > 2  τότε
        γράψε 100
      αλλιώς
        γράψε 200
      εάν_τέλος
    για_τέλος;
    γράψε i
  τέλος_προγράμματος
```

Το πρόγραμμα full_demo ορίζει μια συνάρτηση athroisma(x, y) που επιστρέφει το άθροισμα δύο αριθμών και περιλαμβάνει δηλώσεις, σύνθετες λογικές συνθήκες, και δομές επανάληψης. Αρχικοποιεί μεταβλητές και με βάση μία σύνθετη εάν συνθήκη καλεί τη συνάρτηση athroisma με διαφορετικά ορίσματα. Στη συνέχεια εκτελεί έναν βρόχο όσο που εμφανίζει τις τιμές του i από 0 έως 2, και έναν βρόχο για που εμφανίζει 100 ή 200 ανάλογα με την τιμή του i. Τέλος, τυπώνει την τελική τιμή του i.

intermediate code

```
1: begin_block , athroisma , _ , _
2: + , x , y , t@1
3: retv , t@1 , _ , _
4: end_block , athroisma , _ , _
5: begin_block , full_demo , _ , _
6: := , 2 , _ , a
7: := , 3 , _ , b
8: := , 1 , _ , c
9: := , 20 , _ , d
10: := , 0 , _ , e
11: < , a , b , 13
12: jump , _ , _ , 15
13: = , c , d , 19
14: jump , _ , _ , 15
15: > , b , a , 17
16: jump , _ , _ , 25
17: = , e , 0 , 19
18: jump , _ , _ , 25
19: par , a , CV , _
20: par , b , CV , _
21: par , t@2 , RET , _
22: call , athroisma , _ , _
23: := , t@2 , _ , sum
24: jump , _ , _ , 30
25: par , b , CV , _
26: par , d , CV , _
27: par , t@3 , RET , _
28: call , athroisma , _ , _
29: := , t@3 , _ , sum
30: out , sum , _ , _
31: := , 0 , _ , i
32: < , i , 3 , 34
33: jump , _ , _ , 38
34: out , i , _ , _
35: + , i , 1 , t@4
36: := , t@4 , _ , i
37: jump , _ , _ , 32
38: := , 1 , _ , i
39: <= , i , 3 , 41
40: jump , _ , _ , 48
41: + , i , 1 , i
42: > , i , 2 , 44
43: jump , _ , _ , 46
44: out , 100 , _ , _
45: jump , _ , _ , 47
46: out , 200 , _ , _
47: jump , _ , _ , 39
48: out , i , _ , _
49: halt , _ , _ , _
50: end_block , full_demo , _ , _
```

- **Γραμμές 1–4**: Ορίζεται η συνάρτηση athroisma. Εκτελεί πρόσθεση των παραμέτρων x και y και επιστρέφει το αποτέλεσμα με retv.
- **Γραμμές 5–10**: Εκτελείται η αρχικοποίηση των μεταβλητών του κυρίως προγράμματος (a, b, c, d, e).
- **Γραμμές 11–18**: Ελέγχονται οι λογικές συνθήκες του εάν, που περιλαμβάνουν συνδυασμό και/ή. Αν η συνθήκη ισχύει, μεταβαίνουμε στην κλήση athroisma(a, b).
- **Γραμμές 19–23**: Κλήση της συνάρτησης athroisma με παραμέτρους a και b, το αποτέλεσμα επιστρέφεται στη sum.
- **Γραμμές 24–29**: Εναλλακτική κλήση της athroisma με b και d αν δεν ισχύει η αρχική συνθήκη.
- **Γραμμή 30**: Εκτύπωση του sum.
- **Γραμμές 31–37**: Υλοποίηση βρόχου όσο i < 3, με εμφάνιση της i και αύξησή της κατά 1.
- **Γραμμές 38–48**: Υλοποίηση βρόχου για i := 1 έως 3, με εάν στο εσωτερικό: εμφανίζει 100 αν i > 2, αλλιώς 200.
- **Γραμμή 48**: Εκτύπωση τελικής τιμής της i.
- **Γραμμές 49–50**: Τερματισμός του προγράμματος με halt και κλείσιμο του block.

symbol table

```
Symbol Table:

Scope 1:
func : athroisma
x : Type: var , Offset: 12
y : Type: var , Offset: 16
t@1 : Type: temp , Offset: 20

Scope 0:
prog : full_demo
i : Type: var , Offset: 12
a : Type: var , Offset: 16
b : Type: var , Offset: 20
c : Type: var , Offset: 24
d : Type: var , Offset: 28
e : Type: var , Offset: 32
sum : Type: var , Offset: 36
res : Type: var , Offset: 40
athroisma : Type: func, FrameLength: 24,startQuad: 1 , in
t@2 : Type: temp , Offset: 44
t@3 : Type: temp , Offset: 48
t@4 : Type: temp , Offset: 52
```

Scope 1 – Συνάρτηση athroisma

Αυτό το scope αντιστοιχεί στη συνάρτηση athroisma. Περιλαμβάνει:

- **func : athroisma**
  Αναγνωρίζεται ως συνάρτηση.

- **x**: Μεταβλητή εισόδου της συνάρτησης

    - Τύπος: var
    - Offset: 12
- **y**: Δεύτερη μεταβλητή εισόδου
    - Τύπος: var
    - Offset: 16
- **t@1**: Ενδιάμεση (temp) μεταβλητή που χρησιμοποιείται για την αποθήκευση του αποτελέσματος x + y.
    - Τύπος: temp
    - Offset: 20

Scope 0 – Κύριο Πρόγραμμα full_demo

Περιέχει όλες τις μεταβλητές και την αναγνώριση της συνάρτησης:

- **prog : full_demo**
  Το κυρίως πρόγραμμα.
- **i, a, b, c, d, e, sum, res**: Κανονικές μεταβλητές του προγράμματος με συνεχόμενα offsets (12 έως 40 ανά 4 bytes).
  Όλες είναι τύπου var.
- **athroisma**:
  Καταχωρείται ως συνάρτηση τύπου func
    - FrameLength: 24 (μέγεθος ενεργής περιοχής της athroisma)
    - startQuad: 1 (το τετράπλευρο όπου ξεκινά η συνάρτηση)
    - in: δηλώνει οτι η συνάρτηση δέχεται είσοδο με τιμή
- **t@2, t@3, t@4**:
  Temp μεταβλητές που χρησιμοποιούνται στο κυρίως πρόγραμμα για την αποθήκευση ενδιάμεσων αποτελεσμάτων:
    - t@2: επιστροφή από athroisma(a, b)
    - t@3: επιστροφή από athroisma(b, d)
    - t@4: για i + 1 στον βρόχο

assembly

```
.data
   str_nl: .asciz "\n"
   .text

j Lmain
athroisma:
sw ra,(sp)
L_2:
lw t1,-12(sp)
lw t2,-16(sp)
add t1,t1,t2
sw t1,-20(sp)
L_3:
lw t1,-20(sp)
lw t0,-8(sp)
sw t1,(t0)
L_4:
lw ra,(sp)
jr ra
Lmain:
addi sp,sp,-52
addi gp,sp,0
L_6:
li t1,2
sw t1,-16(sp)
L_7:
li t1,3
sw t1,-20(sp)
L_8:
li t1,1
sw t1,-24(sp)
L_9:
li t1,20
sw t1,-28(sp)
L_10:
li t1,0
sw t1,-32(sp)
L_11:
lw t1,-16(sp)
lw t2,-20(sp)
blt t1,t2,L_13
L_12:
j L_15
L_13:
lw t1,-24(sp)
lw t2,-28(sp)
beq t1,t2,L_19
L_14:
j L_15
L_15:
lw t1,-20(sp)
lw t2,-16(sp)
bgt t1,t2,L_17
L_16:
j L_25
L_30:
li a7,1
lw t0,-36(sp)
addi a0,t0,0
ecall
la a0, str_nl
   li a7, 4
   ecall
L_31:
li t1,0
sw t1,-12(sp)
L_32:
lw t1,-12(sp)
li t2,3
blt t1,t2,L_34
L_33:
j L_38
L_34:
li a7,1
lw t0,-12(sp)
addi a0,t0,0
ecall
la a0, str_nl
   li a7, 4
   ecall
L_35:
lw t1,-12(sp)
li t2,1
add t1,t1,t2
sw t1,-52(sp)
L_36:
lw t1,-52(sp)
sw t1,-12(sp)
L_37:
j L_32
L_38:
li t1,1
sw t1,-12(sp)
L_39:
lw t1,-12(sp)
li t2,3
ble t1,t2,L_41
L_40:
j L_48
L_41:
lw t1,-12(sp)
li t2,1
add t1,t1,t2
sw t1,-12(sp)
L_42:
lw t1,-12(sp)
li t2,2
bgt t1,t2,L_44
L_43:
j L_46
```

```
L_17:
lw t1,-32(sp)
li t2,0
beq t1,t2,L_19
L_18:
j L_25
L_19:
addi fp, sp,0
lw t1,-16(sp)
sw t1,-12(fp)
L_20:
lw t1,-20(sp)
sw t1,-16(fp)
L_21:
addi t1,sp,-44
sw t1,-8(fp)
L_22:
sw sp,-4(fp)
addi sp,sp,0
jal athroisma
addi sp,sp,-0
L_23:
lw t1,-44(sp)
sw t1,-36(sp)
L_24:
j L_30
L_25:
lw t1,-20(sp)
sw t1,-20(fp)
L_26:
lw t1,-28(sp)
sw t1,-24(fp)
L_27:
addi t1,sp,-48
sw t1,-8(fp)
L_28:
sw sp,-4(fp)
addi sp,sp,0
jal athroisma
addi sp,sp,-0
L_29:
lw t1,-48(sp)
sw t1,-36(sp)
```

```
L_44:
li a7,1
li t0,100
addi a0,t0,0
ecall
la a0, str_nl
  li a7, 4
  ecall
L_45:
j L_47
L_46:
li a7,1
li t0,200
addi a0,t0,0
ecall
la a0, str_nl
  li a7, 4
  ecall
L_47:
j L_39
L_48:
li a7,1
lw t0,-12(sp)
addi a0,t0,0
ecall
la a0, str_nl
  li a7, 4
  ecall
L_49:
li a7,10
ecall
la a0, str_nl
  li a7, 4
  ecall
L_50:
lw ra,(sp)
jr ra
```

Αποτέλεσμα

```
5
0
1
2
200
100
100
4

-- program is finished running (0) --
```